xjb: Fast Float to String Algorithm

ANONYMOUS* and ANONYMOUS*, Anonymous, Anonymous

Efficiently and accurately converting floating-point numbers to decimal strings is a critical challenge in numerical computation and data exchange. While existing algorithms like Ryū, Dragonbox, and Schubfach satisfy the Steele-White (SW) principle for accuracy, they often suffer from performance bottlenecks due to branch prediction failures and high-precision multiplication overhead. This paper presents a novel floating-point to string conversion algorithm called "xjb", an optimized variant of the Schubfach algorithm designed to deliver superior performance for IEEE754 single-precision (binary32) and double-precision (binary64) floating-point numbers. By minimizing instruction dependencies, reducing multiplication operations, mitigating branch prediction penalties and by utilizing the simd instruction set, xjb achieves significant performance gains. The algorithm features concise core implementation, parallel computing support, and excellent portability and scalability. Extensive benchmarking across diverse platforms, including AMD R7-7840H and Apple M1, demonstrates that xjb outperforms state-of-the-art algorithms in most scenarios while maintaining full compliance with the SW principle.

CCS Concepts: • **Computing methodologies → Representation of mathematical objects**.

Additional Key Words and Phrases: floating-point, printing, algorithm, performance

ACM Reference Format:
Anonymous and Anonymous. 2026. xjb: Fast Float to String Algorithm. *ACM Trans. Program. Lang. Syst.* 37, 4, Article 111 (August 2026), 37 pages. https://doi.org/XXXXXXX.XXXXXXX

1 INTRODUCTION

In 1990, Steele and White [1] published the paper *how to print floating-point numbers Accurately* and proposed the optimal principle of floating-point number printing algorithms (hereinafter referred to as the SW principle) :

- **Information preservation**: The print result can be parsed back to the original floating-point number.
- **Minimum length**: The print result should be as short as possible.
- **Correct rounding**: On the basis of satisfying 1 and 2, if there are two candidate values, they should be correctly rounded (i.e., the even value should be selected).
- **Generate from left to right**: The print result is generated from the left.

Floating-point number printing algorithms that satisfy the SW principle convert floating-point numbers into real values with unique and definite results. Over the past few years, a variety of different algorithms have been proposed, such as Grisu3[2], Errol[3], Ryū[4][5], Schubfach[6], Grisu-Exact[7], Dragonbox[8], and yy_double[9].

*Both authors contributed equally to this research.

Authors' address: Anonymous, Anonymous.com; Anonymous, Anonymous.cn, Anonymous, Anonymous, Anonymous, Anonymous.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM 0164-0925/2026/8-ART111
https://doi.org/XXXXXXX.XXXXXXX

The algorithm in this paper is based on the Schubfach algorithm, and is inspired by algorithms such as yy_double and Dragonbox. This article only introduces two floating-point number types, IEEE754-binary32 and IEEE754-binary64. To simplify the content, in this article, float represents IEEE754-binary32 and double represents IEEE754-binary64. The article altogether contains nine python code files, and in this paper, the algorithm implementation code you can be found at https://github.com/xjb714/xjb.

2 IEEE754 FLOATING POINT NUMBER REPRESENTATION

Since the print result of a negative floating-point number only has one more negative sign than the print result of its absolute value, this article only discusses positive floating-point numbers and does not include special values such as 0, NaN, and Inf.

The IEEE754 double-precision floating-point number consists of 64 bits, including 1 sign bit ($sign$), 11 exponent bits (exp), and 52 fraction bits ($frac$). $sign$'s range is 0 or 1, exp's range is $[0, 2047]$, and $frac$'s range is $\left[0, 2^{52}-1\right]$.

The IEEE754 single-precision floating-point number consists of 32 bits, including 1 sign bit ($sign$), 8 exponent bits (exp), and 23 fraction bits ($frac$). $sign$'s range is 0 or 1, exp's range is $[0, 255]$, and $frac$'s range is $\left[0, 2^{23}-1\right]$.

When $frac = 0$, it is an irregular floating-point number.

The real value of the positive floating-point number v can be expressed as the following expression:

$$\begin{aligned} double{:}v &= \left(frac + \left(exp \neq 0 ? 2^{52} : 0\right)\right) \cdot 2^{\max(exp,1)-1075} = c \cdot 2^q \\ float{:}v &= \left(frac + \left(exp \neq 0 ? 2^{23} : 0\right)\right) \cdot 2^{\max(exp,1)-150} = c \cdot 2^q \end{aligned} \tag{1}$$

We classify floating-point numbers into three categories in total: subnormal, irregular, and normal. When $exp = 0$ (referred to as subnormal floating-point numbers), there is:

$$\text{subnormal} : \begin{cases} double : v = frac \cdot 2^{-1074} \\ float : v = frac \cdot 2^{-149} \end{cases} \tag{2}$$

When $frac = 0$ (referred to as irregular floating-point numbers), there is:

$$\text{irregular} : \begin{cases} double : v = 2^{52} \cdot 2^{exp-1075} \\ float : v = 2^{23} \cdot 2^{exp-150} \end{cases} \tag{3}$$

When $frac \neq 0$ and $exp \neq 0$ (referred to as normal floating-point number), there is:

$$\text{normal} : \begin{cases} double : v = \left(frac + 2^{52}\right) \cdot 2^{exp-1075} \\ float : v = \left(frac + 2^{23}\right) \cdot 2^{exp-150} \end{cases} \tag{4}$$

In the rounding interval R_v of floating-point numbers, all real numbers will be rounded to this floating-point number when parsed. R_v is:

$$\begin{aligned} v_l &= \begin{cases} \left(c - \frac{1}{2}\right) \cdot 2^q, \text{if } frac \neq 0 \text{ or } exp \leqslant 1 \\ \left(c - \frac{1}{4}\right) \cdot 2^q, \text{if } frac = 0 \end{cases} \\ v_r &= \left(c + \frac{1}{2}\right) \cdot 2^q \\ R_v &= \begin{cases} [v_l, v_r], \text{if } frac\%2 = 0 \\ (v_l, v_r), \text{if } frac\%2 = 1 \end{cases} \end{aligned} \tag{5}$$

When $frac \neq 0$, it is a regular floating-point number. The regular floating-point numbers include all subnormal and normal floating-point numbers. When the floating-point number is a regular floating-point number, 2^{q-1} is the rounded radius.

3 PRINCIPLE OF ALGORITHM

At present, other algorithms use a large number of branches, which can easily lead to branch prediction failure penalties and excessive high multiplication overhead. The algorithm in this paper will minimize the overhead of branch prediction failures and reduce the number of multiplication operations to improve performance. Moreover, the core code for the algorithm implementation in this paper is very concise . The process of printing floating-point numbers is usually divided into two parts: the first part is to convert the floating-point number to a decimal number, and the second part is to convert the decimal number to a string.

For the range of double-precision floating-point numbers, there are a total of 2046 valid irregular floating-point numbers (i.e., exp values range from 1 to 2046). Similarly, there are a total of 254 irregular values for the range of single-precision floating-point numbers (i.e., exp values range from 1 to 254).

The valid range for c and q in regular floating-point numbers is:

$$\begin{aligned} float &: \begin{cases} 1 \leqslant c \leqslant 2^{24}-1, c \neq 2^{23}; q = -149 \\ 2^{23}+1 \leqslant c \leqslant 2^{24}-1; -148 \leqslant q \leqslant 104 \end{cases} \\ double &: \begin{cases} 1 \leqslant c \leqslant 2^{53}-1, c \neq 2^{52}; q = -1074 \\ 2^{52}+1 \leqslant c \leqslant 2^{53}-1; -1073 \leqslant q \leqslant 971 \end{cases} \end{aligned} \tag{6}$$

The valid range for c and q in irregular floating-point numbers is:

$$\begin{aligned} float &: \{c = 2^{23}; -149 \leqslant q \leqslant 104 \\ double &: \{c = 2^{52}; -1074 \leqslant q \leqslant 971 \end{aligned} \tag{7}$$

The valid range for c and q in subnormal floating-point numbers is:

$$\begin{aligned} float &: \{c \leqslant 2^{23}-1; q = -149 \\ double &: \{c \leqslant 2^{52}-1; q = -1074 \end{aligned} \tag{8}$$

Floating-point numbers that do not fall within the subnormal range are called normal floating-point numbers.

Regular floating-point numbers account for the vast majority of all possible values of floating-point numbers and are the most worthy of discussion part. Therefore, unless otherwise specified, only regular floating-point numbers will be discussed below.

Suppose the floating-point number v is converted to the optimal solution that satisfies the SW principle as opt, d is a positive integer and k is an integer,which is expressed as:

$$\begin{aligned} &v = c \cdot 2^q \longrightarrow opt = d \cdot 10^k \\ &opt \in R_v; d \in N^+; k \in Z \end{aligned} \tag{9}$$

For example: IEEE754-binary64 floating-point number "1.3", the real value of the floating-point number is 1.3000000000000000444089209850062616169452667236328125, hexadecimal representation of floating-point Numbers is 3ff4cccccccccccd, Then the opt value that meets the SW principle is 1.3. The IEEE754-binary32 floating-point number "1.3" has an actual value of 1.2999999523162841796875, and its hexadecimal representation is 3FA66666. Therefore, the opt value that satisfies the SW principle is 1.3.

3.1 Review the Schubfach algorithm and the derivation of the algorithm in this paper

According to the Schubfach algorithm, the possible values of d can be one of the following four situations:

$$10 \cdot \lfloor v \cdot 10^{-k-1} \rfloor, \lfloor 10 \cdot \left(v \cdot 10^{-k-1}\right) \rfloor, \lfloor 10 \cdot \left(v \cdot 10^{-k-1}\right) \rfloor + 1, 10 \cdot \lfloor v \cdot 10^{-k-1} \rfloor + 10 \tag{10}$$

The calculation method of k in equation (10) is as follows:

$$k = \lfloor q \cdot \lg(2) \rfloor \text{ if } v \in regular \text{ else } \lfloor q \cdot \lg(2) - \lg(\frac{4}{3}) \rfloor \tag{11}$$

In the range of float and double, equation (11) can be equivalent to:

$$\begin{aligned} double : k &= (q \cdot 315653 - (v \in regular ? 0 : 131072)) \gg 20 \\ float : k &= (q \cdot 1233 - (v \in regular ? 0 : 512)) \gg 12 \end{aligned} \tag{12}$$

Suppose the integer part of $v \cdot 10^{-k-1}$ is m and the decimal part is n, then we have:

$$\begin{aligned} &\lfloor v \cdot 10^{-k-1} \rfloor = m \\ &v \cdot 10^{-k-1} = m + n \\ &0 \leqslant n = v \cdot 10^{-k-1} - \lfloor v \cdot 10^{-k-1} \rfloor < 1 \end{aligned} \tag{13}$$

Then the decimal part of $v \cdot 10^{-k}$ is expressed as:

$$v \cdot 10^{-k} - \lfloor v \cdot 10^{-k} \rfloor = 10m + 10n - \lfloor 10m + 10n \rfloor = 10n - \lfloor 10n \rfloor \tag{14}$$

The possible values of d obtained from equation (10) are:

$$10m, \lfloor 10\,(m+n) \rfloor, \lfloor 10\,(m+n) \rfloor + 1, 10m + 10 \tag{15}$$

The possible values of d in equation (15) can be simplified to:

$$10m, 10m + \lfloor 10n \rfloor, 10m + \lfloor 10n \rfloor + 1, 10m + 10 \tag{16}$$

Among them, $10m$ represents the minimum possible value and $10m + 10$ represents the maximum possible value. Suppose ten is used to represent $10m$. There are four possible values for one, with $d = ten + one$, denoted as:

$$\begin{aligned} &ten = 10m \\ &one \in \{0, \lfloor 10n \rfloor, \lfloor 10n \rfloor + 1, 10\} \\ &d = ten + one \end{aligned} \tag{17}$$

Calculating d will be converted to calculating ten and one.

The final possible values of d are as follows:

- $10m$
  When the following conditions are met, the result is $10m$ (or equivalent to $one = 0$). That is, the floating-point number v minus the minimum possible value of $10m$ is less than the rounded radius 2^{q-1}.

$$\begin{aligned} c \cdot 2^q - 10m \cdot 10^k &< 2^{q-1} \\ c \cdot 2^q - \lfloor c \cdot 2^q \cdot 10^{-k-1} \rfloor \cdot 10^{k+1} &< 2^{q-1} \\ c \cdot 2^q \cdot 10^{-k-1} - \lfloor c \cdot 2^q \cdot 10^{-k-1} \rfloor &< 2^{-1} \cdot 2^q \cdot 10^{-k-1} \\ n &< 2^{-1} \cdot 2^q \cdot 10^{-k-1} \\ 2^{-1} \cdot 2^q \cdot 10^{-k-1} &> n \end{aligned} \tag{18}$$

Or when $2^{-1} \cdot 2^q \cdot 10^{-k-1} = n$, $c\%2 = 0$ must also be satisfied. Therefore, the following conditions are valid:

$$\text{if } 2^{-1} \cdot 2^q \cdot 10^{-k-1} > n \text{ or } \left(2^{-1} \cdot 2^q \cdot 10^{-k-1} = n \ \&\& \ c\%2 = 0\right) : one = 0 \tag{19}$$

- $10m + 10$
  When the following conditions are met, the result is $10m + 10$ (or equivalent to $one = 10$). The maximum possible value of $10m + 10$ minus the floating-point number v is less than the rounded radius 2^{q-1}.

$$\begin{aligned} (10m + 10) \cdot 10^k - c \cdot 2^q &< 2^{q-1} \\ \lfloor c \cdot 2^q \cdot 10^{-k-1} \rfloor \cdot 10^{k+1} + 10^{k+1} - c \cdot 2^q &< 2^{q-1} \\ \lfloor c \cdot 2^q \cdot 10^{-k-1} \rfloor - c \cdot 2^q \cdot 10^{-k-1} + 1 &< 2^{-1} \cdot 2^q \cdot 10^{-k-1} \\ 1 - n &< 2^{-1} \cdot 2^q \cdot 10^{-k-1} \\ 2^{-1} \cdot 2^q \cdot 10^{-k-1} &> 1 - n \end{aligned} \tag{20}$$

  Or when $2^{-1} \cdot 2^q \cdot 10^{-k-1} = 1 - n$, $c\%2 = 0$ must also be satisfied. Therefore, the following conditions are valid:

$$\text{if } 2^{-1} \cdot 2^q \cdot 10^{-k-1} > 1 - n \text{ or } \left(2^{-1} \cdot 2^q \cdot 10^{-k-1} = 1 - n \ \&\& \ c\%2 = 0\right) : one = 10 \tag{21}$$

- $10m + \lfloor 10n \rfloor$ **or** $10m + \lfloor 10n \rfloor + 1$
  When none of the conditions are met as $d = 10m$ or $d = 10m + 10$, d is either $10m + \lfloor 10n \rfloor$ or $10m + \lfloor 10n \rfloor + 1$. The final value is determined based on the decimal part of $10n$. If the decimal part is 0.5, it is rounded to the nearest even value; if it is not 0.5, it is rounded to the nearest value. For irregular floating-point numbers, it is also necessary to determine whether $10m + \lfloor 10n \rfloor$ is within the rounding interval R_v. If it is not, then $10m + \lfloor 10n \rfloor + 1$.

In summary, the steps of the Schubfach algorithm variants are as follows process (22), that is, the algorithms proposed in this paper. This algorithm process (22) is applicable to float and double floating-point numbers(xjb32 for float, xjb64 for double). Taking a floating-point number v as input, c and q are extracted, and the calculation results d (line 15) and k (line 2) are returned. The real value represented by the returned results is $d \cdot 10^k$, which conforms to the SW principle. The calculation process of k is relatively simple and can be obtained from (12). Therefore, the following only focuses on introducing the rapid calculation process of d. The following will be divided into five parts to introduce the algorithm process (22) :

(1) Introduce the pre-computation process of the algorithm's lookup table.
(2) Quickly calculate m.
(3) Quickly determine whether $one = 0$ or $one = 10$.
(4) Quickly calculate $\lfloor 10n \rfloor$ and determine whether $one = \lfloor 10n \rfloor$ or $one = \lfloor 10n \rfloor + 1$ based on the decimal part of $10n$.
(5) Processing of irregular floating-point numbers.

The following will discuss the above content in detail from Section 3.2 to Section 3.6.

$$
\begin{aligned}
&\text{input}: c, q \\
&\text{output}: d, k \\
&(\ 1)\ v = c \cdot 2^q \\
&(\ 2)\ k = \lfloor q \cdot \lg(2) \rfloor \text{ if } v \in regular \text{ else } \lfloor q \cdot \lg(2) - \lg(\frac{4}{3}) \rfloor \\
&(\ 3)\ m = \lfloor v \cdot 10^{-k-1} \rfloor, n = v \cdot 10^{-k-1} - m \\
&(\ 4)\ ten = 10m \\
&(\ 5)\ \text{if } 10n - \lfloor 10n \rfloor = 0.5 : one = \lfloor 10n \rfloor \text{ if } (\lfloor 10n \rfloor \% 2 = 0) \text{ else } \lfloor 10n \rfloor + 1 \\
&(\ 6)\ \text{if } 10n - \lfloor 10n \rfloor < 0.5 : one = \lfloor 10n \rfloor \\
&(\ 7)\ \text{if } 10n - \lfloor 10n \rfloor > 0.5 : one = \lfloor 10n \rfloor + 1 \\
&(\ 8)\ \text{if } v \in irregular : \\
&(\ 9)\quad \text{if } 10n - \lfloor 10n \rfloor > 2^{q-2} \cdot 10^{-k} : one = \lfloor 10n \rfloor + 1 \\
&(10)\quad \text{if } 2^{q-2} \cdot 10^{-k-1} \geqslant n : one = 0 \\
&(11)\ \text{else}: \\
&(12)\quad \text{if } 2^{q-1} \cdot 10^{-k-1} > n \text{ or } \left(2^{q-1} \cdot 10^{-k-1} = n \ \&\&\ c\%2 = 0\right) : one = 0 \\
&(13)\ \text{endif} \\
&(14)\ \text{if } 2^{q-1} \cdot 10^{-k-1} > 1 - n \text{ or } \left(2^{q-1} \cdot 10^{-k-1} = 1 - n \ \&\&\ c\%2 = 0\right) : one = 10 \\
&(15)\ d = ten + one
\end{aligned}
\tag{22}
$$

3.2 Pre-computation of Lookup Table

The algorithm in this paper uses a lookup table to store the values of 10^{-k-1} for q in the range of $[-149, 104]$ for float and $[-1074, 971]$ for double. In the algorithm of this paper, float uses 64-bit precision and double uses 128-bit precision lookup tables. The code implementation in this section is gen.py. Suppose the bit length of a single value data in the lookup table is B. For float, it has $B = 64$, and for double, it has $B = 128$. Suppose there are integers e_{10} and real numbers e_2, where $1 \leqslant f < 2$. There are:

$$f \cdot 2^{\lfloor e_2 \rfloor} = 2^{e_2} = 10^{e_{10}} \tag{23}$$

Then:

$$\lfloor e_2 \rfloor = \lfloor e_{10} \cdot \log_2(10) \rfloor \tag{24}$$

The calculation leads to f, and the following conclusions are drawn:

$$f = \frac{10^{e_{10}}}{2^{\lfloor e_{10} \cdot \log_2(10) \rfloor}} \tag{25}$$

The way to calculate the lookup table is as follows (using the upward rounding method) :

$$lookup[e_{10}] = \lceil f \cdot 2^{B-1} \rceil = \lceil \frac{10^{e_{10}}}{2^{\lfloor e_{10} \cdot \log_2(10) \rfloor}} \cdot 2^{B-1} \rceil = \lceil 10^{e_{10}} \cdot 2^{B-1-\lfloor e_{10} \cdot \log_2(10) \rfloor} \rceil \tag{26}$$

For float, when $0 \leqslant e_{10} \leqslant 27$, $f \cdot 2^{B-1}$ is an integer in equation (26). For double, when $0 \leqslant e_{10} \leqslant 55$, $f \cdot 2^{B-1}$ is an integer in equation (26). The detailed calculation process is as follows:

- Float

The range of $-k-1$ is calculated to be [-32, 44] through the q value range in equation (6), so the lookup table contains representation values from 10 to the power of -32 to 10 to the power of 44. The calculation process is as follows:

$$\begin{aligned} &-32 \leqslant e_{10} \leqslant 44 \\ &e_2 = \left|\lfloor e_{10} \cdot \log_2(10) \rfloor - 63\right| \\ &pow10t = \begin{cases} 2^{e_2}//10^{|e10|}; \text{if } e_{10} < 0 \\ 10^{|e10|}//2^{e_2}; \text{if } e_{10} \geqslant 20 \\ 10^{|e10|} \cdot 2^{e_2}; \text{if } 1 \leqslant e_{10} \leqslant 19 \end{cases} \\ &f_{1,e_{10}} = pow10 = pow10t + (e_{10} \geqslant 0 \&\& e_{10} \leqslant 27 ? 0 : 1) \end{aligned} \tag{27}$$

When $0 \leqslant e_{10} \leqslant 27$, the lookup table variable indicates that the values $f_{1,e_{10}} \cdot 2^{\lfloor e_{10} \cdot \log_2(10) \rfloor - 63}$ and $10^{e_{10}}$ are equal. In other cases, the relative error is less than 2^{-63}. Expressed as:

$$\begin{aligned} r_{1,e_{10}} &= \frac{f_{1,e_{10}} \cdot 2^{\lfloor e_{10} \cdot \log_2(10) \rfloor - 63}}{10^{e_{10}}} \\ &\in \begin{cases} 1; \text{if } 0 \leqslant e_{10} \leqslant 27 \\ \left(1, 1 + 2^{-63}\right); \text{if } e_{10} < 0 \text{ or } e_{10} > 27 \end{cases} \end{aligned} \tag{28}$$

- Double
  The range of $-k-1$ is calculated to be [-293, 323] through the q value range in equation (6), so the lookup table contains representation values from 10 to the power of -293 to 10 to the power of 323. The calculation process is as follows:

$$\begin{aligned} &-293 \leqslant e_{10} \leqslant 323 \\ &e_2 = \left|\lfloor e_{10} \cdot \log_2(10) \rfloor - 127\right| \\ &pow10t = \begin{cases} 2^{e_2}//10^{|e10|}; \text{if } e_{10} < 0 \\ 10^{|e10|}//2^{e_2}; \text{if } e_{10} \geqslant 39 \\ 10^{|e10|} \cdot 2^{e_2}; \text{if } 1 \leqslant e_{10} \leqslant 38 \end{cases} \\ &f_{1,e_{10}} = pow10 = pow10t + (e_{10} \geqslant 0 \&\& e_{10} \leqslant 55 ? 0 : 1) \end{aligned} \tag{29}$$

  When $0 \leqslant e_{10} \leqslant 55$, the lookup table variable indicates that the values $f_{1,e_{10}} \cdot 2^{\lfloor e_{10} \cdot \log_2(10) \rfloor - 127}$ and $10^{e_{10}}$ are equal. In other cases, the relative error is less than 2^{-127}. Expressed as:

$$\begin{aligned} r_{1,e_{10}} &= \frac{f_{1,e_{10}} \cdot 2^{\lfloor e_{10} \cdot \log_2(10) \rfloor - 127}}{10^{e_{10}}} \\ &\in \begin{cases} 1; \text{if } 0 \leqslant e_{10} \leqslant 55 \\ \left(1, 1 + 2^{-127}\right); \text{if } e_{10} < 0 \text{ or } e_{10} > 55 \end{cases} \end{aligned} \tag{30}$$

The following uses r_1 to represent all possible errors of the lookup table values within the float range, r_2 to represent all possible errors of the lookup table values within the double range, and r to represent all possible errors of the lookup table values within either the float or double range. In algorithm process (22), an approximate representation value of 10 to the power of $-k-1$ needs to be obtained through a lookup table. From equation (28) and equation (30), the lookup table representation value is error-free when q is within the following range:

$$\begin{aligned} float &: 0 \leqslant -k-1 \leqslant 27 \Rightarrow -93 \leqslant q \leqslant -1 \\ double &: 0 \leqslant -k-1 \leqslant 55 \Rightarrow -186 \leqslant q \leqslant -1 \end{aligned} \tag{31}$$

When q is not within the range of equation (31), the error range of the value represented by the lookup table can be concluded as follows:

$$\begin{aligned} float &: 0 < r_1 - 1 < 2^{-63} \\ double &: 0 < r_2 - 1 < 2^{-127} \end{aligned} \tag{32}$$

The introduction of the lookup table calculation process is complete. The storage space required for a float range lookup table is 616 bytes, and that for a double range lookup table is 9872 bytes.

3.3 Quickly Calculate m

Relevant theorems (partially from the Dragonbox algorithm paper) : Suppose there are positive integers n,P,and Q, where P and Q are coprime, $P < Q$, $1 \leqslant n \leqslant n_{max}$,$Q > n_{max}$,$P^*/Q^*$ is the best rational approximation result greater than or equal to P/Q, P_*/Q_* is the best rational approximation result less than or equal to P/Q, and it satisfies $Q^* \leqslant n_{max}$, $Q_* \leqslant n_{max}$. And if $n \cdot P$ does not divide Q evenly, it is expressed as:

$$\lfloor n \cdot \frac{P}{Q} \rfloor + 1 = \lceil n \cdot \frac{P}{Q} \rceil \tag{33}$$

Suppose the following holds true:

$$\lfloor n \cdot \frac{P}{Q} \rfloor = \lfloor n \cdot \xi \rfloor \tag{34}$$

Then there are:

$$\frac{P_*}{Q_*} = \max_{1 \leqslant n \leqslant n_{\max}} \frac{\lfloor n \cdot \frac{P}{Q} \rfloor}{n} \leqslant \xi < \min_{1 \leqslant n \leqslant n_{\max}} \frac{\lfloor n \cdot \frac{P}{Q} \rfloor + 1}{n} = \min_{1 \leqslant n \leqslant n_{\max}} \frac{\lceil n \cdot \frac{P}{Q} \rceil}{n} = \frac{P^*}{Q^*} \tag{35}$$

Therefore, the range of values for ξ is:

$$\frac{P_*}{Q_*} \leqslant \xi < \frac{P^*}{Q^*} \tag{36}$$

And the range of the decimal part with $n \cdot \frac{P}{Q}$ is:

$$\left[\frac{(Q_* P) \,\%Q}{Q}, \frac{(Q^* P) \,\%Q}{Q} \right] \tag{37}$$

That is, when $n = Q_*$, the decimal part is the smallest; when $n = Q^*$, the decimal part is the largest.

The definition of the best rational approximation function is as follows (this function is implemented on line 15 of the test1.py file):

$$(DN, UP) = f\,(C, P, Q) \tag{38}$$

The function (38) calculate the best rational approximation result with a denominator not exceeding C based on the mean term theorem of the Farey sequence. DN and UP are two adjacent terms in the C-order Farey sequence F_C.

In algorithm process (22), m is calculated as $\lfloor v \cdot 10^{-k-1} \rfloor$ (line 3). Just prove that the following equation holds:

$$m = \lfloor c \cdot 2^q \cdot 10^{-k-1} \rfloor = \lfloor c \cdot 2^q \cdot r \cdot 10^{-k-1} \rfloor \tag{39}$$

Where r is the error of the lookup table value, as defined in equation (28) and equation (30). When the condition (31) is met, r is 1, and the equation (39) clearly holds. When r is not 1, there is:

$$\begin{aligned} float &: 1 < r < 1 + 2^{-63} \\ double &: 1 < r < 1 + 2^{-127} \end{aligned} \tag{40}$$

Calculate the range of $2^q \cdot 10^{-k-1}$ and we get:

$$2^q \cdot 10^{-k-1} = 10^{-1} \cdot \left(2^q \cdot 10^{-\lfloor q \cdot \lg(2) \rfloor}\right) = 10^{-1} \cdot \left(10^{q \cdot \lg(2) - \lfloor q \cdot \lg(2) \rfloor}\right) \tag{41}$$

When q is not 0, equation (41) exists:

$$\begin{aligned} q \cdot \lg(2) &\neq \lfloor q \cdot \lg(2) \rfloor \\ 0 < q \cdot \lg(2) - \lfloor q \cdot \lg(2) \rfloor &< 1 \end{aligned} \tag{42}$$

When q is 0, $q \cdot \lg(2) - \lfloor q \cdot \lg(2) \rfloor = 0$, so the final conclusion is:

$$10^{-1} \leqslant 2^q \cdot 10^{-k-1} < 1 \tag{43}$$

Because there is:

$$c \cdot 2^q \cdot 10^{-k-1} = c \cdot \frac{2^{q-k-1}}{5^{k+1}} \in [0.1c\,, c) \tag{44}$$

Therefore:

$$c \cdot 2^q \cdot 10^{-k-1} = \begin{cases} \frac{c \cdot 2^{q-k-1}}{5^{k+1}}; q \geqslant 1 \\ \frac{c}{2^{1+k-q} \cdot 5^{k+1}} = \frac{c}{10}; q = 0 \\ \frac{c \cdot 5^{-k-1}}{2^{1+k-q}}; q < 0 \end{cases} \tag{45}$$

Suppose:

$$c \cdot 2^q \cdot 10^{-k-1} = c \cdot \frac{x}{y} < c \tag{46}$$

Then there are:

$$(x, y) = \begin{cases} \left(2^{q-k-1}, 5^{k+1}\right); q \geqslant 1 \\ (1, 10); q = 0 \\ \left(5^{-k-1}, 2^{1+k-q}\right); q < 0 \end{cases} \tag{47}$$

Suppose:

$$\begin{aligned} float &: c \leqslant c_{\max} = C_1 = 2^{24} - 1 \\ double &: c \leqslant c_{\max} = C_2 = 2^{53} - 1 \end{aligned} \tag{48}$$

The following is represented by C as C_1 or C_2. C within the float range is C_1, and C within the double range is C_2.

When $y > C$, calculate the P^* and Q^* corresponding to each q by calling $f\,(C, x, y)$ according to function (38). And calculate the minimum BIT value when the following conditions are met:

$$\frac{x}{y}\left(1 + 2^{-BIT}\right) < \frac{P^*}{Q^*} \tag{49}$$

When $y \leqslant C$, there is:

$$c \cdot \frac{x}{y}\left(1 + \frac{1}{Cy}\right) = \frac{cx + \frac{c}{C} \cdot \frac{x}{y}}{y} < \frac{cx + 1}{y} \tag{50}$$

Therefore:

$$\lfloor c \cdot \frac{x}{y} \rfloor = \lfloor c \cdot \frac{x}{y}\left(1 + \frac{1}{Cy}\right) \rfloor \tag{51}$$

Similarly, calculate the minimum BIT value:

$$\frac{x}{y}\left(1 + 2^{-BIT}\right) < \frac{x}{y}\left(1 + \frac{1}{Cy}\right) \tag{52}$$

In summary, the calculation results of the maximum value among the minimum BIT values corresponding to different q are as follows (the running result is in the test1.py file, and the running time of this code is only about 1 to 2 seconds) :

$$\begin{aligned} float : BIT_{\max} &= 52 \\ double : BIT_{\max} &= 113 \end{aligned} \tag{53}$$

Therefore, the following conclusions exist:

$$\begin{aligned} float : \lfloor c \cdot \frac{x}{y} \rfloor &= \lfloor c \cdot \frac{x}{y} \cdot \left(1 + 2^{-52}\right) \rfloor = \lfloor c \cdot \frac{x}{y} \cdot r_1 \rfloor \\ double : \lfloor c \cdot \frac{x}{y} \rfloor &= \lfloor c \cdot \frac{x}{y} \cdot \left(1 + 2^{-113}\right) \rfloor = \lfloor c \cdot \frac{x}{y} \cdot r_2 \rfloor \end{aligned} \tag{54}$$

This section has been verified.After quickly calculating m, the value of $ten = 10m$ can be obtained very quickly.

3.4 Quickly Determine Whether $one = 0$ or $one = 10$

In algorithm process (22), the conditions for determining $one = 0$ and $one = 10$ are on lines 12, and 14. This section will introduce how to quickly determine whether $one = 0$ or $one = 10$ holds by using equivalent conditions.

When discussing the case of $2^{-1} \cdot 2^q \cdot 10^{-k-1} = n$ (line 12, one might be 0), it is equivalent to:

$$\begin{aligned} c \cdot 2^q \cdot 10^{-k-1} - \lfloor c \cdot 2^q \cdot 10^{-k-1} \rfloor &= 2^{-1} \cdot 2^q \cdot 10^{-k-1} \\ (2c - 1) \cdot 2^{q-1} \cdot 10^{-k-1} &= \lfloor c \cdot 2^q \cdot 10^{-k-1} \rfloor \end{aligned} \tag{55}$$

When discussing the case of $2^{-1} \cdot 2^q \cdot 10^{-k-1} = 1 - n$ (line 14, one might be 10), it is equivalent to:

$$\begin{aligned} \lfloor c \cdot 2^q \cdot 10^{-k-1} \rfloor - c \cdot 2^q \cdot 10^{-k-1} + 1 &= 2^{-1} \cdot 2^q \cdot 10^{-k-1} \\ (2c + 1) \cdot 2^{q-1} \cdot 10^{-k-1} &= \lfloor c \cdot 2^q \cdot 10^{-k-1} \rfloor + 1 \end{aligned} \tag{56}$$

Since equation (43), we have:

$$2^{q-1} \cdot 10^{-k-1} \in [0.05\ , 0.5) \tag{57}$$

Therefore, there is:

$$\begin{aligned} \lfloor c \cdot 2^q \cdot 10^{-k-1} \rfloor - 1 &< c \cdot 2^q \cdot 10^{-k-1} - 0.5 \\ &< (2c - 1) \cdot 2^{q-1} \cdot 10^{-k-1} \\ &\leqslant c \cdot 2^q \cdot 10^{-k-1} - 0.05 < \lfloor c \cdot 2^q \cdot 10^{-k-1} \rfloor + 1 \end{aligned} \tag{58}$$

Therefore, for equation (55), when $(2c - 1) \cdot 2^{q-1} \cdot 10^{-k-1}$ is an integer, it must be equal to $\lfloor c \cdot 2^q \cdot 10^{-k-1} \rfloor$. Similarly, for equation (56), there is:

$$\begin{aligned} \lfloor c \cdot 2^q \cdot 10^{-k-1} \rfloor &< c \cdot 2^q \cdot 10^{-k-1} + 0.05 \\ &\leqslant (2c + 1) \cdot 2^{q-1} \cdot 10^{-k-1} \\ &< c \cdot 2^q \cdot 10^{-k-1} + 0.5 < \lfloor c \cdot 2^q \cdot 10^{-k-1} \rfloor + 2 \end{aligned} \tag{59}$$

Therefore, for equation (56), when $(2c + 1) \cdot 2^{q-1} \cdot 10^{-k-1}$ is an integer, it must be equal to $\lfloor c \cdot 2^q \cdot 10^{-k-1} \rfloor + 1$.

In conclusion, it is equivalent to discussing whether $(2c \pm 1) \cdot 2^{q-1} \cdot 10^{-k-1}$ is an integer. There are:

$$(2c \pm 1) \cdot 2^{q-1} \cdot 10^{-k-1} = (2c \pm 1) \cdot 2^{q-k-2} \cdot 5^{-k-1} \tag{60}$$

According to the range of q, there are:

$$\begin{cases} q-k-2 \geqslant 0, -k-1 < 0; q \geqslant 2 \\ q-k-2 < 0, -k-1 < 0; 1 \geqslant q \geqslant 0 \\ q-k-2 < 0, -k-1 \geqslant 0; q < 0 \end{cases} \tag{61}$$

Therefore, equation (60) is equivalent to:

$$(2c \pm 1) \cdot 2^{q-1} \cdot 10^{-k-1} = \begin{cases} \frac{(2c\pm1)\cdot 2^{q-k-2}}{5^{k+1}}; q \geqslant 2 \\ \frac{(2c\pm1)}{2^{2+k-q}\cdot 5^{k+1}}; 1 \geqslant q \geqslant 0 \\ \frac{(2c\pm1)\cdot 5^{-k-1}}{2^{2+k-q}}; q < 0 \end{cases} \tag{62}$$

According to the different ranges of q, the following situations are discussed:

- $q \geqslant 2$
  From $q \geqslant 2$, we get $k \geqslant 0$.When $q \geqslant 2$, it is equivalent to discussing whether $(2c \pm 1)\cdot 2^{q-k-2}$ is divisible by 5^{k+1}. Since 2 and 5 are coprime, it is equivalent to discussing whether $(2c \pm 1)$ is divisible by 5^{k+1}.

  $$(2c \pm 1) \ \ \%5^{k+1} = 0 \tag{63}$$

  Suppose t is a positive integer:

  $$2c \pm 1 = t \cdot 5^{k+1}; t \geqslant 1 \tag{64}$$

  Since $2c \pm 1$ is odd, t is also odd. Because the following conditions exist:

  $$\begin{aligned} float &: 2c-1 \in \left[2^{24}+1, 2^{25}-3\right]; 2c+1 \in \left[2^{24}+3, 2^{25}-1\right]; \\ double &: 2c-1 \in \left[2^{53}+1, 2^{54}-3\right]; 2c+1 \in \left[2^{53}+3, 2^{54}-1\right]; \end{aligned} \tag{65}$$

  Therefore, the following satisfies:

  $$\begin{aligned} float &: 2^{24}+1 \leqslant t \cdot 5^{k+1} \leqslant 2^{25}-1 \\ double &: 2^{53}+1 \leqslant t \cdot 5^{k+1} \leqslant 2^{54}-1 \end{aligned} \tag{66}$$

  Therefore, the following conclusions are drawn:

  $$\begin{aligned} float &: \frac{2^{24}+1}{5^{k+1}} \leqslant t \leqslant \frac{2^{25}-1}{5^{k+1}}; \\ double &: \frac{2^{53}+1}{5^{k+1}} \leqslant t \leqslant \frac{2^{54}-1}{5^{k+1}}; \end{aligned} \tag{67}$$

  For the above equation (67), the maximum value of k when t can obtain at least one odd number is:

  $$\begin{aligned} float &: k_{\max} = 9 \Rightarrow q_{\max} = 33, t = 3 \\ double &: k_{\max} = 22 \Rightarrow q_{\max} = 76, t = 1 \end{aligned} \tag{68}$$

  Therefore, the maximum value of k is 9 within the float range and 22 within the double range. Therefore, when k exceeds the above range, $(2c \pm 1)$ is not divisible by 5^{k+1}.
- $1 \geqslant q \geqslant 0$
  Because the denominator $2^{2+k-q} \cdot 5^{k+1}$ is even and the numerator $(2c \pm 1)$ is odd, the condition is not met.
- $q < 0$
  Because the denominator 2^{2+k-q} is even and the numerator $(2c \pm 1) \cdot 5^{-k-1}$ is odd, the condition is not met.

In summary, the situations when $(2c \pm 1) \cdot 2^{q-1} \cdot 10^{-k-1}$ is an integer are as follows:

$$\begin{aligned} float &: 2 \leqslant q \leqslant 33 \ \&\& \ \ (2c \pm 1)\,\%5^{k+1} = 0; \\ double &: 2 \leqslant q \leqslant 76 \ \&\& \ \ (2c \pm 1)\,\%5^{k+1} = 0; \end{aligned} \tag{69}$$

And, the range of $-k-1$ is:

$$\begin{aligned} float &: -10 \leqslant -k-1 \leqslant -1 \\ double &: -23 \leqslant -k-1 \leqslant -1 \end{aligned} \tag{70}$$

When $2^{-1} \cdot 2^{q} \cdot 10^{-k-1} = n$, the following conclusions can be drawn:

$$\begin{aligned} float &: \left\{2^{35} \cdot 2^{q} \cdot 10^{-k-1} = 2^{36} \cdot n \Rightarrow \lfloor 2^{35} \cdot 2^{q} \cdot 10^{-k-1} \rfloor = \lfloor 2^{36} \cdot n \rfloor \right. \\ double &: \left\{2^{63} \cdot 2^{q} \cdot 10^{-k-1} = 2^{64} \cdot n \Rightarrow \lfloor 2^{63} \cdot 2^{q} \cdot 10^{-k-1} \rfloor = \lfloor 2^{64} \cdot n \rfloor \right. \end{aligned} \tag{71}$$

When $2^{-1} \cdot 2^{q} \cdot 10^{-k-1} = 1-n$, the following conclusions can be drawn:

$$\begin{aligned} float &: \left\{ \begin{array}{c} 2^{35} \cdot 2^{q} \cdot 10^{-k-1} = 2^{36} - 2^{36} \cdot n \Rightarrow \\ \lfloor 2^{35} \cdot 2^{q} \cdot 10^{-k-1} \rfloor = \lfloor 2^{36} - 2^{36} \cdot n \rfloor = 2^{36} - 1 - \lfloor 2^{36} \cdot n \rfloor \end{array} \right. \\ double &: \left\{ \begin{array}{c} 2^{63} \cdot 2^{q} \cdot 10^{-k-1} = 2^{64} - 2^{64} \cdot n \Rightarrow \\ \lfloor 2^{63} \cdot 2^{q} \cdot 10^{-k-1} \rfloor = \lfloor 2^{64} - 2^{64} \cdot n \rfloor = 2^{64} - 1 - \lfloor 2^{64} \cdot n \rfloor \end{array} \right. \end{aligned} \tag{72}$$

The discussion on whether $\lfloor 2^{36} - 2^{36} \cdot n \rfloor = 2^{36} - 1 - \lfloor 2^{36} \cdot n \rfloor$ in equation (72) holds true, that is, whether $2^{36} \cdot n$ in equation (72) is an integer, or equivalent to discussing whether the following values are integers when equation (69) holds true (the same applies to double) :

$$\begin{aligned} float &: 2^{36} \cdot (m+n) = c \cdot 2^{q+36} \cdot 10^{-k-1} = c \cdot 2^{q-k+35} \cdot 5^{-k-1} = c \cdot \frac{2^{q-k+35}}{5^{k+1}} \\ double &: 2^{64} \cdot (m+n) = c \cdot 2^{q+64} \cdot 10^{-k-1} = c \cdot 2^{q-k+63} \cdot 5^{-k-1} = c \cdot \frac{2^{q-k+63}}{5^{k+1}} \end{aligned} \tag{73}$$

Suppose c can divide 5^{k+1} evenly (where t is a temporary integer variable):

$$c = t \cdot 5^{k+1}; t \geqslant 1 \tag{74}$$

Therefore, when equation (74) was established, there were:

$$2c \pm 1 = 2 \cdot t \cdot 5^{k+1} \pm 1 \tag{75}$$

Expression (75) cannot divide 5^{k+1} evenly, which contradicts equation (69), so c cannot divide 5^{k+1} evenly. Therefore, for float, $c \cdot 2^{q+36} \cdot 10^{-k-1}$ and $2^{36} \cdot n$ are not integers; For double, $c \cdot 2^{64+q} \cdot 10^{-k-1}$ and $2^{64} \cdot n$ are not integers, that is:

$$\begin{aligned} float &: \lfloor 2^{36} - 2^{36} \cdot n \rfloor = 2^{36} + \lfloor -2^{36} \cdot n \rfloor = 2^{36} - 1 - \lfloor 2^{36} \cdot n \rfloor \\ double &: \lfloor 2^{64} - 2^{64} \cdot n \rfloor = 2^{64} + \lfloor -2^{64} \cdot n \rfloor = 2^{64} - 1 - \lfloor 2^{64} \cdot n \rfloor \end{aligned} \tag{76}$$

Therefore, the conclusion (72) is correct. Discuss the necessary and sufficient conditions for whether $\lfloor 2^{35} \cdot 2^{q} \cdot 10^{-k-1} \rfloor = \lfloor 2^{36} \cdot n \rfloor$ is $2^{-1} \cdot 2^{q} \cdot 10^{-k-1} = n$. The same applies to double, expressed as:

$$\begin{aligned} float &: 2^{-1} \cdot 2^{q} \cdot 10^{-k-1} = n \Leftrightarrow \lfloor 2^{35} \cdot 2^{q} \cdot 10^{-k-1} \rfloor = \lfloor 2^{36} \cdot n \rfloor \\ double &: 2^{-1} \cdot 2^{q} \cdot 10^{-k-1} = n \Leftrightarrow \lfloor 2^{63} \cdot 2^{q} \cdot 10^{-k-1} \rfloor = \lfloor 2^{64} \cdot n \rfloor \end{aligned} \tag{77}$$

Similarly, the necessary and sufficient conditions for whether $\lfloor 2^{35} \cdot 2^{q} \cdot 10^{-k-1} \rfloor = \lfloor 2^{36} - 2^{36} \cdot n \rfloor$ is $2^{-1} \cdot 2^{q} \cdot 10^{-k-1} = 1 - n$. The same applies to double, expressed as:

$$\begin{aligned} float &: 2^{-1} \cdot 2^{q} \cdot 10^{-k-1} = 1 - n \Leftrightarrow \lfloor 2^{35} \cdot 2^{q} \cdot 10^{-k-1} \rfloor = \lfloor 2^{36} - 2^{36} \cdot n \rfloor \\ double &: 2^{-1} \cdot 2^{q} \cdot 10^{-k-1} = 1 - n \Leftrightarrow \lfloor 2^{63} \cdot 2^{q} \cdot 10^{-k-1} \rfloor = \lfloor 2^{64} - 2^{64} \cdot n \rfloor \end{aligned} \tag{78}$$

The sufficient conditions of equations (77) and (78) are obviously established. Introduce the proof that equation (77) holds. For float, only the necessary conditions need to be discussed, that is, whether $2^{-1} \cdot 2^{q} \cdot 10^{-k-1} = n$ must hold true when $\lfloor 2^{35} \cdot 2^{q} \cdot 10^{-k-1} \rfloor = \lfloor 2^{36} \cdot n \rfloor$ holds, or equivalent to $\lfloor 2^{35} \cdot 2^{q} \cdot 10^{-k-1} \rfloor \neq \lfloor 2^{36} \cdot n \rfloor$ must hold true when $2^{-1} \cdot 2^{q} \cdot 10^{-k-1} \neq n$. The following is proved by proof by contradiction.

Assume that $\lfloor 2^{35} \cdot 2^{q} \cdot 10^{-k-1} \rfloor = \lfloor 2^{36} \cdot n \rfloor$ holds when $2^{-1} \cdot 2^{q} \cdot 10^{-k-1} \neq n$. Then there is:

$$\begin{aligned} &\lfloor 2^{35} \cdot 2^{q} \cdot 10^{-k-1} \rfloor = \lfloor 2^{36} \cdot n \rfloor \\ &\Rightarrow 0 < \left| 2^{35} \cdot 2^{q} \cdot 10^{-k-1} - 2^{36} \cdot n \right| < 1 \\ &\Rightarrow 0 < \left| (2c-1) \cdot 2^{q-1} \cdot 10^{-k-1} - m \right| < 2^{-36} \end{aligned} \tag{79}$$

As is known from equation (58), there is:

$$m - 1 < (2c-1) \cdot 2^{q-1} \cdot 10^{-k-1} < m + 1 \tag{80}$$

Suppose the decimal part of $(2c-1) \cdot 2^{q-1} \cdot 10^{-k-1}$ is represented as n^{-}, thus we have:

$$\left| (2c-1) \cdot 2^{q-1} \cdot 10^{-k-1} - m \right| = \begin{cases} n^{-}; \text{if } (2c-1) \cdot 2^{q-1} \cdot 10^{-k-1} > m \\ 1 - n^{-}; \text{if } (2c-1) \cdot 2^{q-1} \cdot 10^{-k-1} < m \end{cases} \tag{81}$$

Substitute expression (81) into expression (79), and we get:

$$\begin{aligned} &0 < \left| (2c-1) \cdot 2^{q-1} \cdot 10^{-k-1} - m \right| < 2^{-36} \\ &\Rightarrow 0 < n^{-} < 2^{-36} \text{ or } 0 < 1 - n^{-} < 2^{-36} \end{aligned} \tag{82}$$

Similarly, it can be known that the double range is the range of n^{-}. Therefore, there is:

$$\begin{aligned} float &: n^{-} \in \left(0, 2^{-36}\right) \cup \left(1 - 2^{-36}, 1\right) \\ double &: n^{-} \in \left(0, 2^{-64}\right) \cup \left(1 - 2^{-64}, 1\right) \end{aligned} \tag{83}$$

When $2^{-1} \cdot 2^{q} \cdot 10^{-k-1} \neq n$, it is known from equation (55) that $(2c-1) \cdot 2^{q-1} \cdot 10^{-k-1}$ is not an integer. Therefore, there is:

$$0 < n^{-} < 1 \tag{84}$$

It is only necessary to prove that equation (83) does not hold. Discuss the range of the decimal part n^{-} when $(2c-1) \cdot 2^{q-1} \cdot 10^{-k-1}$ is not an integer. According to equation (62), there are:

$$(2c-1) \cdot 2^{q-1} \cdot 10^{-k-1} = (2c-1) \cdot \frac{x}{y} = \begin{cases} \frac{(2c-1) \cdot 2^{q-k-2}}{5^{k+1}}; q \geqslant 2 \\ \frac{(2c-1)}{2^{2+k-q} \cdot 5^{k+1}}; 1 \geqslant q \geqslant 0 \\ \frac{(2c-1) \cdot 5^{-k-1}}{2^{2+k-q}}; q < 0 \end{cases} \tag{85}$$

The maximum value of $2c - 1$ is:

$$\begin{aligned} float &: (2c-1)_{\max} = 2^{25} - 3 \\ double &: (2c-1)_{\max} = 2^{54} - 3 \end{aligned} \tag{86}$$

Discuss based on the denominator range in equation (85).

- $y \leqslant (2c-1)_{\max}$
  When $y \leqslant (2c-1)_{\max}$, $y_{\max}$ is the expression (86), the following holds true:

$$\begin{aligned} \frac{1}{y_{\max}} \leqslant n^- \leqslant 1 - \frac{1}{y_{\max}} \\ \frac{1}{y_{\max}} \leqslant 1 - n^- \leqslant 1 - \frac{1}{y_{\max}} \end{aligned} \tag{87}$$

  Therefore, when $y \leqslant (2c-1)_{\max}$, equation (83) does not hold true.
- $y > (2c-1)_{\max}$
  Call function (38) to calculate the approximation results $P_* \Big/ Q_*$ and $P^* \Big/ Q^*$ of all possible upper and lower limit rational numbers:

$$\left(\frac{P_*}{Q_*}, \frac{P^*}{Q^*}\right) = f\left((2c-1)_{\max}, x, y\right) \tag{88}$$

  Therefore, for n^-, the following conclusion can be drawn from formula (37).

$$n^- \in \left[\frac{(Q_* x)\ \%y}{y}, \frac{(Q^* x)\ \%y}{y}\right] \tag{89}$$

  By exhausting all possibilities, we thus have (the test code file is test3.py) :

$$\begin{aligned} float : 2^{-33} < n^- < 1 - 2^{-29} \\ double : 2^{-62} < n^- < 1 - 2^{-63} \end{aligned} \tag{90}$$

$$\begin{aligned} float : \left[\frac{(Q_* x)\ \%y}{y}, \frac{(Q^* x)\ \%y}{y}\right] \cap \left(0, 2^{-36}\right) = \varnothing \\ \left[\frac{(Q_* x)\ \%y}{y}, \frac{(Q^* x)\ \%y}{y}\right] \cap \left(1 - 2^{-36}, 1\right) = \varnothing \\ double : \left[\frac{(Q_* x)\ \%y}{y}, \frac{(Q^* x)\ \%y}{y}\right] \cap \left(0, 2^{-64}\right) = \varnothing \\ \left[\frac{(Q_* x)\ \%y}{y}, \frac{(Q^* x)\ \%y}{y}\right] \cap \left(1 - 2^{-64}, 1\right) = \varnothing \end{aligned} \tag{91}$$

  Therefore, when $y > (2c-1)_{\max}$, equation (83) does not hold true.

In summary, when $2^{-1} \cdot 2^q \cdot 10^{-k-1} \neq n$, equation (83) does not hold true, that is, $\lfloor 2^{35} \cdot 2^q \cdot 10^{-k-1} \rfloor \neq \lfloor 2^{36} \cdot n \rfloor$ must hold true. Therefore, when $\lfloor 2^{35} \cdot 2^q \cdot 10^{-k-1} \rfloor = \lfloor 2^{36} \cdot n \rfloor$ holds, $2^{-1} \cdot 2^q \cdot 10^{-k-1} = n$ must hold true. Therefore, equation (77) holds.

Similarly, it can be proved that when $\lfloor 2^{35} \cdot 2^q \cdot 10^{-k-1} \rfloor = \lfloor 2^{36} - 2^{36} \cdot n \rfloor$ holds, $2^{-1} \cdot 2^q \cdot 10^{-k-1} = 1 - n$ must hold true. The same applies to double. Similarly, by proof of contradiction, for float, it is assumed that when $2^{-1} \cdot 2^q \cdot 10^{-k-1} \neq 1 - n$ holds, $\lfloor 2^{35} \cdot 2^q \cdot 10^{-k-1} \rfloor = \lfloor 2^{36} - 2^{36} \cdot n \rfloor$ holds. That is:

$$\begin{aligned} &\lfloor 2^{35} \cdot 2^q \cdot 10^{-k-1} \rfloor = \lfloor 2^{36} - 2^{36} \cdot n \rfloor \\ &\Rightarrow 0 < \left| 2^{35} \cdot 2^q \cdot 10^{-k-1} - 2^{36} + 2^{36} \cdot n \right| < 1 \\ &\Rightarrow 0 < \left| 2^{q-1} \cdot 10^{-k-1} - 1 + n \right| < 2^{-36} \\ &\Rightarrow -2^{-36} < (2c+1) \cdot 2^{q-1} \cdot 10^{-k-1} - m - 1 < 2^{-36} \end{aligned} \tag{92}$$

As is known from equation (59), there is:

$$m < (2c+1) \cdot 2^{q-1} \cdot 10^{-k-1} < m+2 \tag{93}$$

Suppose the decimal part of $(2c+1) \cdot 2^{q-1} \cdot 10^{-k-1}$ is represented as n^+, thus we have:

$$(2c+1) \cdot 2^{q-1} \cdot 10^{-k-1} - m - 1 = \begin{cases} n^+; \text{if } (2c+1) \cdot 2^{q-1} \cdot 10^{-k-1} > m+1 \\ 1 - n^+; \text{if } (2c+1) \cdot 2^{q-1} \cdot 10^{-k-1} < m+1 \end{cases} \tag{94}$$

Substitute expression (94) into expression (92), and we get:

$$\begin{aligned} 0 < \left|(2c+1) \cdot 2^{q-1} \cdot 10^{-k-1} - m - 1\right| < 2^{-36} \\ \Rightarrow 0 < 1 - n^+ < 2^{-36} \text{ or } 0 < n^+ < 2^{-36} \end{aligned} \tag{95}$$

Similarly, it can be known that the double range is the range of n^+. Therefore, there is:

$$\begin{aligned} float : n^+ \in \left(0, 2^{-36}\right) \cup \left(1 - 2^{-36}, 1\right) \\ double : n^+ \in \left(0, 2^{-64}\right) \cup \left(1 - 2^{-64}, 1\right) \end{aligned} \tag{96}$$

When $2^{-1} \cdot 2^q \cdot 10^{-k-1} \neq 1 - n$, it is known from equation (56) that $(2c+1) \cdot 2^{q-1} \cdot 10^{-k-1}$ is not an integer. Therefore, there is:

$$0 < n^+ < 1 \tag{97}$$

It is only necessary to prove that equation (96) does not hold. Discuss the range of the decimal part n^+ when $(2c+1) \cdot 2^{q-1} \cdot 10^{-k-1}$ is not an integer. According to equation (62), there are:

$$(2c+1) \cdot 2^{q-1} \cdot 10^{-k-1} = (2c+1) \cdot \frac{x}{y} = \begin{cases} \frac{(2c+1) \cdot 2^{q-k-2}}{5^{k+1}}; q \geqslant 2 \\ \frac{(2c+1)}{2^{2+k-q} \cdot 5^{k+1}}; 1 \geqslant q \geqslant 0 \\ \frac{(2c+1) \cdot 5^{-k-1}}{2^{2+k-q}}; q < 0 \end{cases} \tag{98}$$

The maximum value of $2c+1$ is:

$$\begin{aligned} float : (2c+1)_{\max} = 2^{25} - 1 \\ double : (2c+1)_{\max} = 2^{54} - 1 \end{aligned} \tag{99}$$

Discuss based on the denominator range in equation (98).

- $y \leqslant (2c+1)_{\max}$
  When $y \leqslant (2c+1)_{\max}$, $y_{\max}$ is the expression (99), the following holds true:

$$\begin{aligned} \frac{1}{y_{\max}} \leqslant n^+ \leqslant 1 - \frac{1}{y_{\max}} \\ \frac{1}{y_{\max}} \leqslant 1 - n^+ \leqslant 1 - \frac{1}{y_{\max}} \end{aligned} \tag{100}$$

  Therefore, when $y \leqslant (2c+1)_{\max}$, equation (96) does not hold true.
- $y > (2c+1)_{\max}$
  Call function (38) to calculate the approximation results $\frac{P_*}{Q_*}$ and $\frac{P^*}{Q^*}$ of all possible upper and lower limit rational numbers:

$$\left(\frac{P_*}{Q_*}, \frac{P^*}{Q^*}\right) = f\left((2c+1)_{\max}, x, y\right) \tag{101}$$

  Therefore, for n^+, the following conclusion can be drawn from formula (37).

$$n^+ \in \left[\frac{(Q_* x) \,\%y}{y}, \frac{(Q^* x) \,\%y}{y}\right] \tag{102}$$

By exhausting all possibilities, we thus have (the test code file is test7.py) :

$$\begin{aligned} float &: 2^{-33} < n^{+} < 1 - 2^{-29} \\ double &: 2^{-62} < n^{+} < 1 - 2^{-63} \end{aligned} \tag{103}$$

$$\begin{aligned} float : & \left[\frac{(Q_{*}x)\,\%y}{y}, \frac{(Q^{*}x)\,\%y}{y}\right] \cap \left(0, 2^{-36}\right) = \varnothing \\ & \left[\frac{(Q_{*}x)\,\%y}{y}, \frac{(Q^{*}x)\,\%y}{y}\right] \cap \left(1 - 2^{-36}, 1\right) = \varnothing \\ double : & \left[\frac{(Q_{*}x)\,\%y}{y}, \frac{(Q^{*}x)\,\%y}{y}\right] \cap \left(0, 2^{-64}\right) = \varnothing \\ & \left[\frac{(Q_{*}x)\,\%y}{y}, \frac{(Q^{*}x)\,\%y}{y}\right] \cap \left(1 - 2^{-64}, 1\right) = \varnothing \end{aligned} \tag{104}$$

Therefore, when $y > (2c+1)_{\max}$, equation (96) does not hold true.

In summary, when $2^{-1} \cdot 2^{q} \cdot 10^{-k-1} \neq 1 - n$, equation (96) does not hold true, that is, $\lfloor 2^{35} \cdot 2^{q} \cdot 10^{-k-1} \rfloor \neq \lfloor 2^{36} - 2^{36} \cdot n \rfloor$ must hold true. Therefore, when $\lfloor 2^{35} \cdot 2^{q} \cdot 10^{-k-1} \rfloor = \lfloor 2^{36} - 2^{36} \cdot n \rfloor$ holds, $2^{-1} \cdot 2^{q} \cdot 10^{-k-1} = 1 - n$ must hold true. The same is true for double.Therefore, equation (78) holds.

The following conclusions hold:

$$\begin{aligned} float : \lfloor 2^{36} - 2^{36} \cdot n \rfloor &= \begin{cases} 2^{36} - 1 - \lfloor 2^{36} \cdot n \rfloor; \text{if } c \cdot 2^{36+q} \cdot 10^{-k-1} \notin Z \\ 2^{36} - \lfloor 2^{36} \cdot n \rfloor; \text{if } c \cdot 2^{36+q} \cdot 10^{-k-1} \in Z \end{cases} \\ double : \lfloor 2^{64} - 2^{64} \cdot n \rfloor &= \begin{cases} 2^{64} - 1 - \lfloor 2^{64} \cdot n \rfloor; \text{if } c \cdot 2^{64+q} \cdot 10^{-k-1} \notin Z \\ 2^{64} - \lfloor 2^{64} \cdot n \rfloor; \text{if } c \cdot 2^{64+q} \cdot 10^{-k-1} \in Z \end{cases} \end{aligned} \tag{105}$$

Discuss whether the following equation (106) holds when conditions (69) and (70) are met:

$$\begin{aligned} float : & \lfloor c \cdot \frac{2^{q+35-k}}{5^{k+1}} \rfloor = \lfloor c \cdot \frac{2^{q+35-k}}{5^{k+1}} \cdot r \rfloor \\ & = \lfloor c \cdot \frac{2^{q+35-k}}{5^{k+1}} \cdot \frac{\left(2^{63-\lfloor (-k-1)\cdot \log_2(10) \rfloor} // 10^{k+1}\right) + 1}{10^{-k-1}} \cdot 2^{\lfloor (-k-1)\cdot \log_2(10) \rfloor - 63} \rfloor \\ double : & \lfloor c \cdot \frac{2^{q+63-k}}{5^{k+1}} \rfloor = \lfloor c \cdot \frac{2^{q+63-k}}{5^{k+1}} \cdot r \rfloor \\ & = \lfloor c \cdot \frac{2^{q+63-k}}{5^{k+1}} \cdot \frac{\left(2^{127-\lfloor (-k-1)\cdot \log_2(10) \rfloor} // 10^{k+1}\right) + 1}{10^{-k-1}} \cdot 2^{\lfloor (-k-1)\cdot \log_2(10) \rfloor - 127} \rfloor \end{aligned} \tag{106}$$

There are:

$$\begin{aligned} float : & \lfloor c \cdot \frac{2^{q+35-k}}{5^{k+1}} \rfloor = \lfloor 2^{36} \cdot (m+n) \rfloor = 2^{36} \cdot m + \lfloor 2^{36} \cdot n \rfloor \\ double : & \lfloor c \cdot \frac{2^{q+63-k}}{5^{k+1}} \rfloor = \lfloor 2^{64} \cdot (m+n) \rfloor = 2^{64} \cdot m + \lfloor 2^{64} \cdot n \rfloor \end{aligned} \tag{107}$$

It has been proven earlier that m can be accurately calculated. Then, when (106) holds true, the values $\lfloor 2^{36} \cdot n \rfloor$ and $\lfloor 2^{64} \cdot n \rfloor$ on the right side of equations (71) and (72) can be accurately calculated.

From equation (64), we have:

$$c = \frac{t \cdot 5^{k+1} - 1}{2} \tag{108}$$

Substituting equation (108) into equation (106), we have:

$$\begin{aligned} float: c \cdot \frac{2^{q+35-k}}{5^{k+1}} &= t \cdot 2^{q+34-k} - \frac{2^{q+34-k}}{5^{k+1}} \\ double: c \cdot \frac{2^{q+63-k}}{5^{k+1}} &= t \cdot 2^{q+62-k} - \frac{2^{q+62-k}}{5^{k+1}} \end{aligned} \tag{109}$$

When conditions (69) and (70) are met, $t \cdot 2^{q+34-k}$ and $t \cdot 2^{q+62-k}$ are integers. Under the condition of meeting condition (69), the decimal part of expression (109) is represented as:

$$\begin{aligned} float: \frac{2^{q+34-k} \% 5^{k+1}}{5^{k+1}}; 2 \leqslant q \leqslant 33 \\ double: \frac{2^{q+62-k} \% 5^{k+1}}{5^{k+1}}; 2 \leqslant q \leqslant 76 \end{aligned} \tag{110}$$

It is only necessary to prove that the increase in the value $c \cdot \frac{2^{q+35-k}}{5^{k+1}} \cdot r$ on the right side of the expression compared to the value $c \cdot \frac{2^{q+35-k}}{5^{k+1}}$ on the left side plus the decimal part of the value on the left side is less than 1 for equation (106) to hold true. That is:

$$\begin{aligned} float: \frac{2^{q+34-k} \% 5^{k+1}}{5^{k+1}} + \left(c \cdot \frac{2^{q+35-k}}{5^{k+1}} \cdot r - c \cdot \frac{2^{q+35-k}}{5^{k+1}} \right) < 1 \\ double: \frac{2^{q+62-k} \% 5^{k+1}}{5^{k+1}} + \left(c \cdot \frac{2^{q+63-k}}{5^{k+1}} \cdot r - c \cdot \frac{2^{q+63-k}}{5^{k+1}} \right) < 1 \end{aligned} \tag{111}$$

By exhaustionally calculating the maximum possible c value under each q and substituting it into equation (111), it holds. The calculation result is in test2.py. The calculation results show that for the float range and the double range, equation (111) always holds true. Therefore, equation (106) holds true, and thus the values of $\lfloor 2^{36} \cdot n \rfloor$ and $\lfloor 2^{64} \cdot n \rfloor$ on the right side of equations (71) and (72) can be accurately calculated. The values of $\lfloor 2^{35} \cdot 2^{q} \cdot 10^{-k-1} \rfloor$ and $\lfloor 2^{63} \cdot 2^{q} \cdot 10^{-k-1} \rfloor$ on the left side of equations (71) and (72) can be calculated through lookup tables.

$$\begin{aligned} float: \lfloor 2^{35} \cdot 2^{q} \cdot 10^{-k-1} \rfloor = pow10 \gg \left(28 - q - \lfloor (-k-1) \cdot \log_2(10) \rfloor\right) \\ double: \lfloor 2^{63} \cdot 2^{q} \cdot 10^{-k-1} \rfloor = pow10 \gg \left(64 - q - \lfloor (-k-1) \cdot \log_2(10) \rfloor\right) \end{aligned} \tag{112}$$

The code file for verifying the validity of equation (112) is test4.py. Therefore, when conditions (69) and (70) are met, the values of both sides of equations (71) and (72) can be accurately calculated.

Discuss the relationship between the following two values within all ranges of floating-point numbers:

$$\begin{aligned} float: \lfloor c \cdot 2^{q+36} \cdot 10^{-k-1} \rfloor; \lfloor c \cdot 2^{q+36} \cdot r \cdot 10^{-k-1} \rfloor; \\ double: \lfloor c \cdot 2^{q+64} \cdot 10^{-k-1} \rfloor; \lfloor c \cdot 2^{q+64} \cdot r \cdot 10^{-k-1} \rfloor; \end{aligned} \tag{113}$$

When $r = 1$, it is obvious that the two values in expression (113) are equal. When $r \neq 1$, or equivalent to $r > 1$, has:

$$\begin{aligned}
&float: \\
c \cdot 2^{q+36} \cdot r \cdot 10^{-k-1} &= c \cdot 2^{q+36} \cdot 10^{-k-1} + c \cdot 2^{q+36} \cdot (r-1) \cdot 10^{-k-1} \\
&< c \cdot 2^{q+36} \cdot 10^{-k-1} + 2^{24} \cdot 2^{36} \cdot 2^{q} \cdot 10^{-k-1} \cdot (r-1) \\
&< c \cdot 2^{q+36} \cdot 10^{-k-1} + 2^{-3} \\
\lfloor c \cdot 2^{q+36} \cdot r \cdot 10^{-k-1} \rfloor &\leqslant \lfloor c \cdot 2^{q+36} \cdot 10^{-k-1} \rfloor + 1 \\
&double: \\
c \cdot 2^{q+64} \cdot r \cdot 10^{-k-1} &= c \cdot 2^{q+64} \cdot 10^{-k-1} + c \cdot 2^{q+64} \cdot (r-1) \cdot 10^{-k-1} \\
&< c \cdot 2^{q+64} \cdot 10^{-k-1} + 2^{53} \cdot 2^{64} \cdot 2^{q} \cdot 10^{-k-1} \cdot (r-1) \\
&< c \cdot 2^{q+64} \cdot 10^{-k-1} + 2^{-10} \\
\lfloor c \cdot 2^{q+64} \cdot r \cdot 10^{-k-1} \rfloor &\leqslant \lfloor c \cdot 2^{q+64} \cdot 10^{-k-1} \rfloor + 1
\end{aligned} \tag{114}$$

Therefore, there is:

$$\begin{aligned}
float &: 0 \leqslant \lfloor c \cdot 2^{q+36} \cdot r \cdot 10^{-k-1} \rfloor - \lfloor c \cdot 2^{q+36} \cdot 10^{-k-1} \rfloor \leqslant 1 \\
double &: 0 \leqslant \lfloor c \cdot 2^{q+64} \cdot r \cdot 10^{-k-1} \rfloor - \lfloor c \cdot 2^{q+64} \cdot 10^{-k-1} \rfloor \leqslant 1
\end{aligned} \tag{115}$$

Because there is:

$$\lfloor c \cdot 2^{q} \cdot 10^{-k-1} \rfloor = \lfloor c \cdot 2^{q} \cdot r \cdot 10^{-k-1} \rfloor = m \tag{116}$$

$$\begin{aligned}
float &: \lfloor c \cdot 2^{q+36} \cdot 10^{-k-1} \rfloor = 2^{36} \cdot m + \lfloor 2^{36} \cdot n \rfloor \\
double &: \lfloor c \cdot 2^{q+64} \cdot 10^{-k-1} \rfloor = 2^{64} \cdot m + \lfloor 2^{64} \cdot n \rfloor
\end{aligned} \tag{117}$$

Suppose:

$$n_r = c \cdot 2^{q} \cdot r \cdot 10^{-k-1} - m \tag{118}$$

Therefore, the following conclusion can be drawn: when condition (69) is met, from equation (106), we have:

$$\begin{aligned}
float &: 2 \leqslant q \leqslant 33 \,\&\&\, (2c \pm 1) \,\%5^{k+1} = 0 \Rightarrow \lfloor 2^{36} \cdot n \rfloor = \lfloor 2^{36} \cdot n_r \rfloor \\
double &: 2 \leqslant q \leqslant 76 \,\&\&\, (2c \pm 1) \,\%5^{k+1} = 0 \Rightarrow \lfloor 2^{64} \cdot n \rfloor = \lfloor 2^{64} \cdot n_r \rfloor
\end{aligned} \tag{119}$$

Within the range of floating-point numbers, there exists:

$$\begin{aligned}
float &: \lfloor 2^{36} \cdot n \rfloor \leqslant \lfloor 2^{36} \cdot n_r \rfloor \leqslant \lfloor 2^{36} \cdot n \rfloor + 1 \\
double &: \lfloor 2^{64} \cdot n \rfloor \leqslant \lfloor 2^{64} \cdot n_r \rfloor \leqslant \lfloor 2^{64} \cdot n \rfloor + 1
\end{aligned} \tag{120}$$

To simplify the expression, *even* is used to indicate whether c is an even number:

$$even = (c+1)\%2 \in \{0, 1\} \tag{121}$$

When $2^{-1} \cdot 2^{q} \cdot 10^{-k-1} = n$ or $2^{-1} \cdot 2^{q} \cdot 10^{-k-1} = 1 - n$, $2^{-1} \cdot 2^{q} \cdot 10^{-k-1} = n$ is the boundary condition for $one = 0$, and $2^{-1} \cdot 2^{q} \cdot 10^{-k-1} = 1 - n$ is the boundary condition for $one = 10$. Whether *one* is 0 or 10 is determined based on whether c is an even number.Therefore, the following exists:

$$\begin{aligned}
float &: \begin{cases} one = 0 : \lfloor 2^{q+35} \cdot 10^{-k-1} \rfloor + even > \lfloor 2^{36} \cdot n_r \rfloor \\ one = 10 : \lfloor 2^{q+35} \cdot 10^{-k-1} \rfloor + even > 2^{36} - 1 - \lfloor 2^{36} \cdot n_r \rfloor \end{cases} \\
double &: \begin{cases} one = 0 : \lfloor 2^{q+63} \cdot 10^{-k-1} \rfloor + even > \lfloor 2^{64} \cdot n_r \rfloor \\ one = 10 : \lfloor 2^{q+63} \cdot 10^{-k-1} \rfloor + even > 2^{64} - 1 - \lfloor 2^{64} \cdot n_r \rfloor \end{cases}
\end{aligned} \tag{122}$$

Therefore, when $2^{-1} \cdot 2^q \cdot 10^{-k-1} = n$ or $2^{-1} \cdot 2^q \cdot 10^{-k-1} = 1 - n$, we can use the condition (123) to determine whether $one = 0$ or $one = 10$.

$$\begin{aligned} float &: \begin{cases} \text{if } \lfloor 2^{q+35} \cdot 10^{-k-1} \rfloor + even > \lfloor 2^{36} \cdot n_r \rfloor : one = 0 \\ \text{if } \lfloor 2^{q+35} \cdot 10^{-k-1} \rfloor + even > 2^{36} - 1 - \lfloor 2^{36} \cdot n_r \rfloor : one = 10 \end{cases} \\ double &: \begin{cases} \text{if } \lfloor 2^{q+63} \cdot 10^{-k-1} \rfloor + even > \lfloor 2^{64} \cdot n_r \rfloor : one = 0 \\ \text{if } \lfloor 2^{q+63} \cdot 10^{-k-1} \rfloor + even > 2^{64} - 1 - \lfloor 2^{64} \cdot n_r \rfloor : one = 10 \end{cases} \end{aligned} \tag{123}$$

When $2^{-1} \cdot 2^q \cdot 10^{-k-1} > n$ or $2^{-1} \cdot 2^q \cdot 10^{-k-1} > 1 - n$, We can also use the above condition (123) to determine whether $one = 0$ or $one = 10$. When $2^{-1} \cdot 2^q \cdot 10^{-k-1} < n$ or $2^{-1} \cdot 2^q \cdot 10^{-k-1} < 1 - n$, we can also use the above condition (123) to determine whether $one \neq 0$ or $one \neq 10$. There are a total of four situations.The proof is as follows:

(1)When $2^{-1} \cdot 2^q \cdot 10^{-k-1} < n$, there must exist $one \neq 0$, and there is:

$$\begin{aligned} float &: 2^{-1} \cdot 2^q \cdot 10^{-k-1} - n = n^- - 1 \in \left(2^{-33} - 1, -2^{-29}\right) \\ double &: 2^{-1} \cdot 2^q \cdot 10^{-k-1} - n = n^- - 1 \in \left(2^{-62} - 1, -2^{-63}\right) \end{aligned} \tag{124}$$

Therefore, the following exists:

$$\begin{aligned} float &: 2^{q+35} \cdot 10^{-k-1} - 2^{36} \cdot n \in \left(2^3 - 2^{36}, -2^7\right) \\ double &: 2^{q+63} \cdot 10^{-k-1} - 2^{64} \cdot n \in \left(4 - 2^{64}, -2\right) \end{aligned} \tag{125}$$

Suppose there are two real numbers a and b, and the following relationship must exist:

$$\begin{aligned} 0 \leqslant & b - \lfloor b \rfloor < 1 \\ a - \lfloor a \rfloor - 1 < & b - \lfloor b \rfloor < 1 + a - \lfloor a \rfloor \\ a - b - 1 < & \lfloor a \rfloor - \lfloor b \rfloor < a - b + 1 \end{aligned} \tag{126}$$

When $a = 2^{q+35} \cdot 10^{-k-1}$ and $b = 2^{36} \cdot n$ or $a = 2^{q+63} \cdot 10^{-k-1}$ and $b = 2^{64} \cdot n$, the following exists:

$$\begin{aligned} float &: \lfloor 2^{q+35} \cdot 10^{-k-1} \rfloor - \lfloor 2^{36} \cdot n \rfloor < 2^{q+35} \cdot 10^{-k-1} - 2^{36} \cdot n + 1 \\ double &: \lfloor 2^{q+63} \cdot 10^{-k-1} \rfloor - \lfloor 2^{64} \cdot n \rfloor < 2^{q+63} \cdot 10^{-k-1} - 2^{64} \cdot n + 1 \end{aligned} \tag{127}$$

From equation (125), we have:

$$\begin{aligned} float &: \lfloor 2^{q+35} \cdot 10^{-k-1} \rfloor - \lfloor 2^{36} \cdot n \rfloor < 1 - 2^7 < 0 \\ double &: \lfloor 2^{q+63} \cdot 10^{-k-1} \rfloor - \lfloor 2^{64} \cdot n \rfloor < 1 - 2 < 0 \end{aligned} \tag{128}$$

Therefore, there is:

$$\begin{aligned} float &: \lfloor 2^{q+35} \cdot 10^{-k-1} \rfloor + even \leqslant \lfloor 2^{q+35} \cdot 10^{-k-1} \rfloor + 1 \\ &\qquad < \lfloor 2^{36} \cdot n \rfloor \leqslant \lfloor 2^{36} \cdot n_r \rfloor \\ &\Rightarrow \lfloor 2^{q+35} \cdot 10^{-k-1} \rfloor + even < \lfloor 2^{36} \cdot n_r \rfloor \\ double &: \lfloor 2^{q+63} \cdot 10^{-k-1} \rfloor + even \leqslant \lfloor 2^{q+63} \cdot 10^{-k-1} \rfloor + 1 \\ &\qquad < \lfloor 2^{64} \cdot n \rfloor \leqslant \lfloor 2^{64} \cdot n_r \rfloor \\ &\Rightarrow \lfloor 2^{q+63} \cdot 10^{-k-1} \rfloor + even < \lfloor 2^{64} \cdot n_r \rfloor \end{aligned} \tag{129}$$

Therefore, when $2^{-1} \cdot 2^q \cdot 10^{-k-1} < n$, the condition (123) can be used to determine that $one \neq 0$.

(2)When $2^{-1} \cdot 2^q \cdot 10^{-k-1} > n$, there must exist $one = 0$, and there is:

$$\begin{aligned} float &: 2^{-1} \cdot 2^q \cdot 10^{-k-1} - n = n^- \in \left(2^{-33}, 1 - 2^{-29}\right) \\ double &: 2^{-1} \cdot 2^q \cdot 10^{-k-1} - n = n^- \in \left(2^{-62}, 1 - 2^{-63}\right) \end{aligned} \tag{130}$$

Therefore, the following exists:

$$\begin{aligned} float &: 2^{q+35} \cdot 10^{-k-1} - 2^{36} \cdot n \in \left(2^3, 2^{36} - 2^7\right) \\ double &: 2^{q+63} \cdot 10^{-k-1} - 2^{64} \cdot n \in \left(4, 2^{64} - 2\right) \end{aligned} \tag{131}$$

When $a = 2^{q+35} \cdot 10^{-k-1}$ and $b = 2^{36} \cdot n$ or $a = 2^{q+63} \cdot 10^{-k-1}$ and $b = 2^{64} \cdot n$, from equation (126), the following exists:

$$\begin{aligned} float &: \lfloor 2^{q+35} \cdot 10^{-k-1} \rfloor - \lfloor 2^{36} \cdot n \rfloor > 2^{q+35} \cdot 10^{-k-1} - 2^{36} \cdot n - 1 \\ double &: \lfloor 2^{q+63} \cdot 10^{-k-1} \rfloor - \lfloor 2^{64} \cdot n \rfloor > 2^{q+63} \cdot 10^{-k-1} - 2^{64} \cdot n - 1 \end{aligned} \tag{132}$$

From equation (131), we have:

$$\begin{aligned} float &: \lfloor 2^{q+35} \cdot 10^{-k-1} \rfloor - \lfloor 2^{36} \cdot n \rfloor > 2^3 - 1 \geqslant 0 \\ double &: \lfloor 2^{q+63} \cdot 10^{-k-1} \rfloor - \lfloor 2^{64} \cdot n \rfloor > 4 - 1 \geqslant 0 \end{aligned} \tag{133}$$

Therefore, there is:

$$\begin{aligned} float &: \lfloor 2^{q+35} \cdot 10^{-k-1} \rfloor + even \geqslant \lfloor 2^{q+35} \cdot 10^{-k-1} \rfloor \\ &\qquad > \lfloor 2^{36} \cdot n \rfloor + 1 \geqslant \lfloor 2^{36} \cdot n_r \rfloor \\ &\Rightarrow \lfloor 2^{q+35} \cdot 10^{-k-1} \rfloor + even > \lfloor 2^{36} \cdot n_r \rfloor \\ double &: \lfloor 2^{q+63} \cdot 10^{-k-1} \rfloor + even \geqslant \lfloor 2^{q+63} \cdot 10^{-k-1} \rfloor \\ &\qquad > \lfloor 2^{64} \cdot n \rfloor + 1 \geqslant \lfloor 2^{64} \cdot n_r \rfloor \\ &\Rightarrow \lfloor 2^{q+63} \cdot 10^{-k-1} \rfloor + even > \lfloor 2^{64} \cdot n_r \rfloor \end{aligned} \tag{134}$$

Therefore, when $2^{-1} \cdot 2^q \cdot 10^{-k-1} > n$, the condition (123) can be used to determine that $one = 0$.
(3)When $2^{-1} \cdot 2^q \cdot 10^{-k-1} < 1 - n$, there must exist $one \neq 10$, and there is:

$$\begin{aligned} float &: 2^{-1} \cdot 2^q \cdot 10^{-k-1} + n = n^+ \in \left(2^{-33}, 1 - 2^{-29}\right) \\ double &: 2^{-1} \cdot 2^q \cdot 10^{-k-1} + n = n^+ \in \left(2^{-62}, 1 - 2^{-63}\right) \end{aligned} \tag{135}$$

Therefore, the following exists:

$$\begin{aligned} float &: 2^{q+35} \cdot 10^{-k-1} + 2^{36} \cdot n \in \left(2^3, 2^{36} - 2^7\right) \\ double &: 2^{q+63} \cdot 10^{-k-1} + 2^{64} \cdot n \in \left(4, 2^{64} - 2\right) \end{aligned} \tag{136}$$

Suppose there are two real numbers a and b, and the following relationship must exist:

$$\begin{aligned} a - 1 < \lfloor a \rfloor &\leqslant a \\ b - 1 < \lfloor b \rfloor &\leqslant b \\ a + b - 2 < \lfloor a \rfloor + \lfloor b \rfloor &\leqslant a + b \end{aligned} \tag{137}$$

When $a = 2^{q+35} \cdot 10^{-k-1}$ and $b = 2^{36} \cdot n$ or $a = 2^{q+63} \cdot 10^{-k-1}$ and $b = 2^{64} \cdot n$, the following exists:

$$\begin{aligned} float &: \lfloor 2^{q+35} \cdot 10^{-k-1} \rfloor + \lfloor 2^{36} \cdot n \rfloor \leqslant 2^{q+35} \cdot 10^{-k-1} + 2^{36} \cdot n \\ double &: \lfloor 2^{q+63} \cdot 10^{-k-1} \rfloor + \lfloor 2^{64} \cdot n \rfloor \leqslant 2^{q+63} \cdot 10^{-k-1} + 2^{64} \cdot n \end{aligned} \tag{138}$$

From equation (136), we have:

$$\begin{aligned} float &: \lfloor 2^{q+35} \cdot 10^{-k-1} \rfloor + \lfloor 2^{36} \cdot n \rfloor < 2^{36} - 2^7 \\ double &: \lfloor 2^{q+63} \cdot 10^{-k-1} \rfloor + \lfloor 2^{64} \cdot n \rfloor < 2^{64} - 2 \end{aligned} \tag{139}$$

Therefore, there is:

$$\begin{aligned}
float: \lfloor 2^{q+35} \cdot 10^{-k-1} \rfloor + even &\leqslant \lfloor 2^{q+35} \cdot 10^{-k-1} \rfloor + 1 \\
&< 2^{36} - 2 - \lfloor 2^{36} \cdot n \rfloor \\
&< 2^{36} - 1 - \lfloor 2^{36} \cdot n_r \rfloor \\
\Rightarrow \lfloor 2^{q+35} \cdot 10^{-k-1} \rfloor + even &< 2^{36} - 1 - \lfloor 2^{36} \cdot n_r \rfloor \\
double: \lfloor 2^{q+63} \cdot 10^{-k-1} \rfloor + even &\leqslant \lfloor 2^{q+63} \cdot 10^{-k-1} \rfloor + 1 \\
&< 2^{64} - 2 - \lfloor 2^{64} \cdot n \rfloor \\
&< 2^{64} - 1 - \lfloor 2^{64} \cdot n_r \rfloor \\
\Rightarrow \lfloor 2^{q+63} \cdot 10^{-k-1} \rfloor + even &< 2^{64} - 1 - \lfloor 2^{64} \cdot n_r \rfloor
\end{aligned} \tag{140}$$

Therefore, when $2^{-1} \cdot 2^q \cdot 10^{-k-1} < 1 - n$, the condition (123) can be used to determine that $one \neq 10$.
(4)When $2^{-1} \cdot 2^q \cdot 10^{-k-1} > 1 - n$, there must exist $one = 10$, and there is:

$$\begin{aligned}
float: 2^{-1} \cdot 2^q \cdot 10^{-k-1} + n = n^+ + 1 &\in \left(1 + 2^{-33}, 2 - 2^{-29}\right) \\
double: 2^{-1} \cdot 2^q \cdot 10^{-k-1} + n = n^+ + 1 &\in \left(1 + 2^{-62}, 2 - 2^{-63}\right)
\end{aligned} \tag{141}$$

Therefore, the following exists:

$$\begin{aligned}
float: 2^{q+35} \cdot 10^{-k-1} + 2^{36} \cdot n &\in \left(2^3 + 2^{36}, 2^{37} - 2^7\right) \\
double: 2^{q+63} \cdot 10^{-k-1} + 2^{64} \cdot n &\in \left(4 + 2^{64}, 2^{65} - 2\right)
\end{aligned} \tag{142}$$

When $a = 2^{q+35} \cdot 10^{-k-1}$ and $b = 2^{36} \cdot n$ or $a = 2^{q+63} \cdot 10^{-k-1}$ and $b = 2^{64} \cdot n$, from equation (137), the following exists:

$$\begin{aligned}
float: \lfloor 2^{q+35} \cdot 10^{-k-1} \rfloor + \lfloor 2^{36} \cdot n \rfloor &> 2^{q+35} \cdot 10^{-k-1} + 2^{36} \cdot n - 2 \\
double: \lfloor 2^{q+63} \cdot 10^{-k-1} \rfloor + \lfloor 2^{64} \cdot n \rfloor &> 2^{q+63} \cdot 10^{-k-1} + 2^{64} \cdot n - 2
\end{aligned} \tag{143}$$

From equation (142), we have:

$$\begin{aligned}
float: \lfloor 2^{q+35} \cdot 10^{-k-1} \rfloor + \lfloor 2^{36} \cdot n \rfloor &> 2^{36} + 2^3 - 2 > 2^{36} \\
double: \lfloor 2^{q+63} \cdot 10^{-k-1} \rfloor + \lfloor 2^{64} \cdot n \rfloor &> 2^{64} + 2 - 2 > 2^{64}
\end{aligned} \tag{144}$$

Therefore, there is:

$$\begin{aligned}
float: \lfloor 2^{q+35} \cdot 10^{-k-1} \rfloor + even &\geqslant \lfloor 2^{q+35} \cdot 10^{-k-1} \rfloor \\
&> 2^{36} - \lfloor 2^{36} \cdot n \rfloor \\
&> 2^{36} - 1 - \lfloor 2^{36} \cdot n_r \rfloor \\
\Rightarrow \lfloor 2^{q+35} \cdot 10^{-k-1} \rfloor + even &> 2^{36} - 1 - \lfloor 2^{36} \cdot n_r \rfloor \\
double: \lfloor 2^{q+63} \cdot 10^{-k-1} \rfloor + even &\geqslant \lfloor 2^{q+63} \cdot 10^{-k-1} \rfloor \\
&> 2^{64} - \lfloor 2^{64} \cdot n \rfloor \\
&> 2^{64} - 1 - \lfloor 2^{64} \cdot n_r \rfloor \\
\Rightarrow \lfloor 2^{q+63} \cdot 10^{-k-1} \rfloor + even &> 2^{64} - 1 - \lfloor 2^{64} \cdot n_r \rfloor
\end{aligned} \tag{145}$$

Therefore, when $2^{-1} \cdot 2^q \cdot 10^{-k-1} > 1 - n$, the condition (123) can be used to determine that $one = 10$.

From the above proof, it can be seen that when condition (69) is met, the condition (123) can be used to determine whether $one = 0$ or $one = 10$ when $2^{-1} \cdot 2^q \cdot 10^{-k-1} = n$ or $2^{-1} \cdot 2^q \cdot 10^{-k-1} = 1 - n$.

When $2^{-1} \cdot 2^q \cdot 10^{-k-1} > n$ or $2^{-1} \cdot 2^q \cdot 10^{-k-1} > 1 - n$, the condition (123) can be used to determine whether $one = 0$ or $one = 10$. When $2^{-1} \cdot 2^q \cdot 10^{-k-1} < n$ or $2^{-1} \cdot 2^q \cdot 10^{-k-1} < 1 - n$, the condition (123) can be used to determine whether $one \neq 0$ or $one \neq 10$.

The proof process of this section is completed. In the code implementation, the two judgment conditions can be quickly calculated using addition and subtraction shift operations, and can be compiled by the compiler into cmov instructions, thereby reducing the impact of branch prediction failure on performance.

3.5 Determine whether $one = \lfloor 10n \rfloor$ or $one = \lfloor 10n \rfloor + 1$

Determine whether one is $\lfloor 10n \rfloor$ or $\lfloor 10n \rfloor + 1$ based on the decimal part of $10n$. There are two cases: the decimal part of $10n$ is 0.5 and it is not 0.5.

3.5.1 $10n - \lfloor 10n \rfloor = 0.5$. When the decimal part of $10n$ is 0.5, there must be:

$$\begin{aligned} 10n - \lfloor 10n \rfloor &= 0.5 \\ \Rightarrow 10 \cdot c \cdot 2^q \cdot 10^{-k-1} - \lfloor 10 \cdot c \cdot 2^q \cdot 10^{-k-1} \rfloor &= 0.5 \\ \Rightarrow c \cdot 2^q \cdot 10^{-k} - \lfloor c \cdot 2^q \cdot 10^{-k} \rfloor &= 0.5 \\ \Rightarrow c \cdot 2^q \cdot 10^{-k} &= \lfloor c \cdot 2^q \cdot 10^{-k} \rfloor + 0.5 \\ \Rightarrow 2c \cdot 2^q \cdot 10^{-k} &= 2\lfloor c \cdot 2^q \cdot 10^{-k} \rfloor + 1 \end{aligned} \tag{146}$$

So $2c \cdot 2^q \cdot 10^{-k}$ is an odd number. Then the following expression is odd:

$$c \cdot 2^{q+1} \cdot 10^{-k} = c \cdot 2^{q-k+1} \cdot 5^{-k} \tag{147}$$

According to the range of q, there are:

$$c \cdot 2^{q+1} \cdot 10^{-k} = \begin{cases} \frac{c \cdot 2^{q-k+1}}{5^k}; q \geqslant 0 \\ c \cdot 2 \cdot 5^{-k}; q = -1 \\ \frac{c \cdot 5^{-k}}{2^{k-q-1}}; q \leqslant -2 \end{cases} \tag{148}$$

According to the range of q, the following situations are discussed:

- $q \geqslant 0$
  When $q \geqslant 0$, it can be concluded that $q - k + 1 \geqslant 1$, the numerator $c \cdot 2^{q-k+1}$ is even and the denominator 5^k is odd, which does not meet the condition.
- $q = -1$
  When $q = -1$, it can be concluded that $c \cdot 2 \cdot 5^{-k}$ is even, which does not meet the condition.
- $q \leqslant -2$
  5^{-k} is an odd number. c is an odd multiple of 2^{k-q-1}. So:

  $$\begin{aligned} float : c \geqslant 2^{k-q-1} \Rightarrow k - q - 1 \leqslant 22 \Rightarrow q \geqslant -34 \\ double : c \geqslant 2^{k-q-1} \Rightarrow k - q - 1 \leqslant 51 \Rightarrow q \geqslant -75 \end{aligned} \tag{149}$$

  Therefore, when q meets the above conditions, c must be an odd multiple of 2^{k-q-1} to meet the condition. Therefore, when the following conditions are met, expression (147) is an odd number:

  $$\begin{aligned} float : -34 \leqslant q \leqslant -2 \;\&\&\; c\%2^{k-q} = 2^{k-q-1} \\ double : -75 \leqslant q \leqslant -2 \;\&\&\; c\%2^{k-q} = 2^{k-q-1} \end{aligned} \tag{150}$$

  When q is within the above range (150), $r = 1$ is derived from equation (31).Therefore, there is:

  $$n_r = n \tag{151}$$

The following equation holds:

$$20m + 20n = c \cdot 2^q \cdot 10^{-k+1} = c \cdot 2^{q-k+1} \cdot 5^{-k} = \frac{c}{2^{k-q-1}} \cdot 5^{-k} \tag{152}$$

Since $-k \geqslant 1$, 5^{-k} is multiple of 5 and is an odd number. Since $\frac{c}{2^{k-q-1}}$ and 5^{-k} are both odd numbers, $20m$ is an even number, $20n$ is multiple of 5 and is an odd number. Therefore, there is:

$$\begin{aligned} 20n &\in \{5, 15\} \\ \Rightarrow n &\in \{0.25, 0.75\} \\ \Rightarrow n_r &\in \{0.25, 0.75\} \end{aligned} \tag{153}$$

The result of *one* is an even number between $\lfloor 10n \rfloor$ and $\lfloor 10n \rfloor + 1$. Therefore, when the following conditions are met:

$$one = \begin{cases} \lfloor 10n \rfloor = 2, \text{if } n = 0.25 \\ \lfloor 10n \rfloor + 1 = 8, \text{if } n = 0.75 \end{cases} \Rightarrow one = \lfloor 20n + 1 \rfloor // 2 - (n = 0.25 \,?1 : 0) \tag{154}$$

3.5.2 $10n - \lfloor 10n \rfloor \neq 0.5$. When the decimal part of $10n$ is not 0.5, round to the nearest integer value based on the decimal part of $10n$. Therefore, there is:

$$one = \begin{cases} \lfloor 10n \rfloor, \text{if } 10n - \lfloor 10n \rfloor < 0.5 \\ \lfloor 10n \rfloor + 1, \text{if } 10n - \lfloor 10n \rfloor > 0.5 \end{cases} \Rightarrow one = \lfloor 10n + 0.5 \rfloor = \lfloor 20n + 1 \rfloor // 2 \tag{155}$$

Since $\lfloor 20n + 1 \rfloor = \lfloor 20n \rfloor + 1$, it is only necessary to accurately calculate the value of $\lfloor 20n \rfloor$. And, there is:

$$\begin{aligned} d &= ten + one \\ &= 10m + \lfloor 20n + 1 \rfloor // 2 \\ &= (\lfloor 20m + 20n \rfloor + 1) // 2 \end{aligned} \tag{156}$$

Suppose there are:

$$20m + 20n = c \cdot 2^{q+1} \cdot 10^{-k} = c \cdot 2^{q-k+1} \cdot 5^{-k} = c \cdot \frac{x}{y} \tag{157}$$

Suppose the decimal part of $20n$ is n_{20}.

When $y \leqslant c_{\max} = C$, the range of the decimal part must include:

$$\begin{aligned} float &: \frac{1}{2^{24} - 1} = \frac{1}{C} \leqslant n_{20} \leqslant 1 - \frac{1}{C} = \frac{2^{24} - 2}{2^{24} - 1} \\ double &: \frac{1}{2^{53} - 1} = \frac{1}{C} \leqslant n_{20} \leqslant 1 - \frac{1}{C} = \frac{2^{53} - 2}{2^{53} - 1} \end{aligned} \tag{158}$$

When $y > c_{\max} = C$, the range of the decimal part must include(the test file is test5.py):

$$\begin{aligned} float &: 2^{-32} < n_{20} < 1 - 2^{-30} \\ double &: 2^{-64} < n_{20} < 1 - 2^{-62} \end{aligned} \tag{159}$$

Therefore, the range of n_{20} satisfies equation (159). In the code implementation, for float, only the high 36 bits of n_r are retained, and for double, only the high 70 bits of n_r are retained. Suppose the discarded part of a float is represented as n_{36}, and similarly, the discarded part of a double is represented as n_{70}. Therefore, there is:

$$\begin{aligned} float &: n_{36} \in \left[0, 2^{-36}\right) \\ double &: n_{70} \in \left[0, 2^{-70}\right) \end{aligned} \tag{160}$$

Calculate the boundary conditions of the following expression:

$$\begin{aligned} float : F &= 20 \cdot \left(c \cdot 2^q \cdot r \cdot 10^{-k-1} - n_{36}\right) \\ double : F &= 20 \cdot \left(c \cdot 2^q \cdot r \cdot 10^{-k-1} - n_{70}\right) \end{aligned} \tag{161}$$

Therefore, there is:

$$\begin{aligned} float : F_{\min} &> 20 \cdot \left(c \cdot 2^q \cdot 10^{-k-1} - 2^{-36}\right) \\ &= 20m + 20n - 20 \cdot 2^{-36} \\ F_{\max} &< 20 \cdot \left(c \cdot 2^q \cdot \left(1 + 2^{-63}\right) \cdot 10^{-k-1} - 0\right) \\ &< 20m + 20n + 20 \cdot 2^{-63} \cdot c \\ &< 20m + \lfloor 20n \rfloor + 1 \\ double : F_{\min} &> 20 \cdot \left(c \cdot 2^q \cdot 10^{-k-1} - 2^{-70}\right) \\ &= 20m + 20n - 20 \cdot 2^{-70} \\ &> 20m + \lfloor 20n \rfloor \\ F_{\max} &< 20 \cdot \left(c \cdot 2^q \cdot \left(1 + 2^{-127}\right) \cdot 10^{-k-1} - 0\right) \\ &< 20m + 20n + 20 \cdot 2^{-127} \cdot c \\ &< 20m + \lfloor 20n \rfloor + 1 \end{aligned} \tag{162}$$

Therefore, there is:

$$\begin{aligned} float : \lfloor F \rfloor &= 20m + \lfloor 20n \rfloor \\ double : \lfloor F \rfloor &= 20m + \lfloor 20n \rfloor \end{aligned} \tag{163}$$

In fact, in the above proof process, for float, $\lfloor F_{min} \rfloor \neq 20m + \lfloor 20n \rfloor$ may exist, but the code implementation has passed the exhaustive test, so this not-so-perfect proof process can be ignored. Therefore, the calculation of d can be simplified as follows:

$$\begin{aligned} d &= ten + one \\ &= (\lfloor F \rfloor + 1)//2 \\ &= (\lfloor 20 \cdot (c \cdot 2^q \cdot r \cdot 10^{-k-1} - n_x) \rfloor + 1)//2 \end{aligned} \tag{164}$$

For the float range, $n_x = n_{36}$; for the double range, $n_x = n_{70}$.

For double, quickly determine that $n == 0.25$ in equation (154).
When $n = 0.25$,$\lfloor 2^{64} \cdot n_r \rfloor = \lfloor 2^{64} \cdot n \rfloor = 2^{62}$. Therefore, the following condition can be used to quickly determine whether $n = 0.25$:

$$double : n = 0.25 \text{ if } \lfloor 2^{64} \cdot n_r \rfloor = 2^{62} \tag{165}$$

When $n \neq 0.25$,Calculate the range of the decimal part of the following expression:

$$4m + 4n = c \cdot 2^{q+2} \cdot 10^{-k-1} \tag{166}$$

Therefore, when equation (166) is not an integer, we have(the test file is test6.py):

$$2^{-62} < 4n - \lfloor 4n \rfloor < 1 - 2^{-62} \tag{167}$$

Calculate the two boundary cases of $4n$ that are closest to 1:

$$\begin{aligned} \lfloor 4n \rfloor = 0 \Rightarrow 4n - 0 < 1 - 2^{-62} \Rightarrow \lfloor 2^{64} \cdot n \rfloor \leqslant 2^{62} - 2 \\ \lfloor 4n \rfloor = 1 \Rightarrow 4n - 1 > 2^{-62} \Rightarrow \lfloor 2^{64} \cdot n \rfloor \geqslant 2^{62} + 1 \end{aligned} \tag{168}$$

Then there are:

$$\begin{aligned} \lfloor 2^{64} \cdot n \rfloor \neq 2^{62} \,\&\&\, \lfloor 2^{64} \cdot n \rfloor + 1 \neq 2^{62} \\ \Rightarrow \lfloor 2^{64} \cdot n_r \rfloor \neq 2^{62} \end{aligned} \tag{169}$$

Therefore, the following condition can be used to quickly determine whether $n \neq 0.25$:

$$double : n \neq 0.25 \text{ if } \lfloor 2^{64} \cdot n_r \rfloor \neq 2^{62} \tag{170}$$

In summary, for double, the following condition can be used to quickly determine whether $n = 0.25$:

$$\begin{aligned} double : n = 0.25 \text{ if } \lfloor 2^{64} \cdot n_r \rfloor = 2^{62} \\ double : n \neq 0.25 \text{ if } \lfloor 2^{64} \cdot n_r \rfloor \neq 2^{62} \end{aligned} \tag{171}$$

In the double range, introduce another faster way to calculate *one*:

$$double : one = \lfloor \frac{\lfloor 2^{64} \cdot n_r \rfloor}{2^{64}} \cdot 10 + \left((n = 0.25) ? 0 : \left(2^{-1} + \frac{6}{2^{64}} \right) \right) \rfloor \tag{172}$$

The proof of equation (172) is as follows:
when $n = 0.25$, $\lfloor \frac{\lfloor 2^{64} \cdot n_r \rfloor}{2^{64}} \cdot 10 \rfloor = \lfloor 10n \rfloor = 2$;
when $n \neq 0.25$, equation (172) can be equivalent to the following:

$$double : one = \lfloor \frac{\lfloor 2^{64} \cdot n_r \rfloor}{2^{64}} \cdot 10 + 2^{-1} + \frac{6}{2^{64}} \rfloor \tag{173}$$

According to the $10n - \lfloor 10n \rfloor$ range, *one* is represented as:

$$double : one = \begin{cases} \lfloor 10n \rfloor, \text{if } 10n - \lfloor 10n \rfloor < 0.5 \\ 8, \text{if } 10n - \lfloor 10n \rfloor = 0.5 \\ \lfloor 10n \rfloor + 1, \text{if } 10n - \lfloor 10n \rfloor > 0.5 \end{cases} = \lfloor 20n + 1 \rfloor // 2 \tag{174}$$

Therefore,when $n \neq 0.25$, we need to prove that the following equation holds:

$$\lfloor \frac{\lfloor 2^{64} \cdot n_r \rfloor}{2^{64}} \cdot 10 + 2^{-1} + \frac{6}{2^{64}} \rfloor = \begin{cases} \lfloor 10n \rfloor, \text{if } 10n - \lfloor 10n \rfloor < 0.5 \\ 8, \text{if } 10n - \lfloor 10n \rfloor = 0.5 \\ \lfloor 10n \rfloor + 1, \text{if } 10n - \lfloor 10n \rfloor > 0.5 \end{cases} = \lfloor 20n + 1 \rfloor // 2 \tag{175}$$

From the range of n, there is:

$$\frac{\lfloor 2^{64} \cdot n_r \rfloor}{2^{64}} \in \left(n_r - 2^{-64}, n_r \right] \tag{176}$$

Because the following conditions exist:

$$\begin{aligned} c \cdot 2^q \cdot 10^{-k-1} = m + n \\ c \cdot 2^q \cdot r \cdot 10^{-k-1} = m + n_r \end{aligned} \tag{177}$$

Therefore, the following relationship can be concluded:

$$\begin{aligned} n_r - n &= (r-1)\cdot c\cdot 2^q\cdot 10^{-k-1} \\ n_r &= (r-1)\cdot(m+n)+n \\ \Rightarrow n \leqslant n_r &< 2^{-127}\cdot c + n \\ n \leqslant n_r &< 2^{-127}\cdot 2^{53} + n \\ n \leqslant n_r &< 2^{-74} + n \end{aligned} \tag{178}$$

From equation (176) and (178), it can be concluded that:

$$\begin{aligned} \frac{\lfloor 2^{64}\cdot n_r\rfloor}{2^{64}} &\in \left(n - 2^{-64}, n + 2^{-74}\right) \\ \Rightarrow \frac{\lfloor 2^{64}\cdot n_r\rfloor}{2^{64}}\cdot 10 &\in \left(10n - 10\cdot 2^{-64}, 10n + 10\cdot 2^{-74}\right) \\ \Rightarrow \frac{\lfloor 2^{64}\cdot n_r\rfloor}{2^{64}}\cdot 20 &\in \left(20n - 20\cdot 2^{-64}, 20n + 20\cdot 2^{-74}\right) \\ \Rightarrow \frac{\lfloor 2^{64}\cdot n_r\rfloor}{2^{64}}\cdot 20 &\in \left(\lfloor 20n\rfloor + n_{20} - 20\cdot 2^{-64}, \lfloor 20n\rfloor + n_{20} + 20\cdot 2^{-74}\right) \end{aligned} \tag{179}$$

Discuss the range of values of x when the following conditions are met.

$$\lfloor \frac{\lfloor 2^{64}\cdot n_r\rfloor}{2^{64}}\cdot 20 + 1 + x\rfloor // 2 = \lfloor 20n+1\rfloor // 2 = one \tag{180}$$

Therefore, the following conclusions can be drawn:

$$\begin{aligned} \lfloor 20n\rfloor + n_{20} - 20\cdot 2^{-64} + 1 + x \geqslant \lfloor 20n+1\rfloor &\Rightarrow x \geqslant 20\cdot 2^{-64} - n_{20} \\ \lfloor 20n\rfloor + n_{20} + 20\cdot 2^{-74} + 1 + x < \lfloor 20n+2\rfloor &\Rightarrow x < 1 - 20\cdot 2^{-74} - n_{20} \end{aligned} \tag{181}$$

Suppose $x = 12\cdot 2^{-64}$. Through the exhaustive method, all floating-point numbers that do not meet the following conditions can be obtained.

$$x = 12\cdot 2^{-64} \geqslant 20\cdot 2^{-64} - n_{20} \tag{182}$$

All floating-point numbers that do not meet condition (182) are as follows (in hexadecimal) :

$$\begin{aligned} &0xd17c0747bd76fa1, \\ &0xd27c0747bd76fa1, \\ &0x4d73de005bd620df, \\ &0x4d83de005bd620df, \\ &0x4d93de005bd620df, \end{aligned} \tag{183}$$

Through the exhaustive method, all floating-point numbers that do not meet the following conditions can be obtained.

$$x = 12\cdot 2^{-64} < 1 - 20\cdot 2^{-74} - n_{20} \tag{184}$$

All floating-point numbers that do not meet condition (184) are as follows (in hexadecimal) :

$$\begin{aligned} &0x612491daad0ba280, \\ &0x6159b651584e8b20, \\ &0x619011f2d73116f4, \\ &0x61c4166f8cfd5cb1, \\ &0x61d4166f8cfd5cb1, \end{aligned} \tag{185}$$

There are:

$$2(\frac{\lfloor 2^{64} \cdot n_r \rfloor}{2^{64}} \cdot 10 + 2^{-1} + \frac{6}{2^{64}}) = \frac{\lfloor 2^{64} \cdot n_r \rfloor}{2^{64}} \cdot 20 + 1 + x \tag{186}$$

When the floating-point number is not within the above range (183) and (185), the condition (181) is satisfied. We have tested all floating-point numbers within the above-mentioned range (183) and (185), and the algorithm implementation code has output the correct result, that is, it satisfies the SW principle. The test process file is test8.py.

In summary, equation (175) and equation (172) holds. Therefore, equation (172) can be used to quickly calculate *one*.

3.6 Irregular Number

Due to the limited and small number of irregular floating-point numbers, there are a total of 2046 double floating-point numbers and 254 float floating-point numbers. The correctness of the algorithm code in this paper can be proved by the exhaustive method. Therefore, it is not introduced in this article. For the specific implementation process, please refer to the source code.

3.7 implementation of pseudocode

This section presents the pseudo-code calculation process for regular floating-point numbers. This section does not cover the calculation process of irregular floating-point numbers. For the detailed calculation process, please refer to the source code.

For single-precision floating-point numbers, the process of calculating d and k is as follows:

$$\begin{aligned}
\text{uint32 } vi &= bit\ copy\ from\ \text{f32 } v \\
\text{uint64 } c, \text{int32 } q &= \ extract\ from\ vi \\
\text{const int } BIT &= 36 \\
\text{const uint64 } offset &= (1ULL \ll (BIT - 2)) - 7 \\
\text{int32 } k &= (q \cdot 1233) \gg 12 \\
\text{int32 } h &= q + ((1701 \cdot (-k - 1)) \gg 9) \\
\text{uint64 } pow10 &= \text{get_pow10}(-k - 1) \\
\text{uint64 } cb &= c \ll (h + BIT + 1) \\
\text{uint64 } hi64 &= \text{umul64_64_hi64}\,(cb, pow10) \\
\text{bool } even &= (vi + 1)\,\&1 \\
\text{uint64 } half &= (pow10 \gg (65 - (h + BIT + 1))) + even \\
\text{uint32 } shorter &= (hi64 + half) \gg BIT \\
\text{uint64 } bias &= (hi64 \gg (BIT - 4))\,\&15 \\
\text{uint32 } longer &= (5 \cdot hi64 + offset + bias) \gg (BIT - 1) \\
\text{bool } updown &= shorter > ((hi64 - half) \gg BIT) \\
\text{uint32 } d &= updown?shorter * 10 : longer
\end{aligned} \tag{187}$$

The c value can have a maximum of 23 bits of valid information, while the $pow10$ value has 64 bits of valid information. Therefore, when performing multiplication, the result will have at most 87 bits of valid value. It can be concluded that the range of h is [-4, -1]. Therefore, cb will not overflow and will retain all the bit information of c. When $e10 = -k - 1$, the value of $pow10$ is calculated using Equation (27). From Equation (28), it can be concluded that:

$$pow10 \cdot 2^{\lfloor (-k-1) \cdot \log 2(10) \rfloor - 63} = 10^{-k-1} \cdot r_{1,-k-1} \tag{188}$$

From equation (54), the following conclusion can be drawn:

$$\begin{aligned} m &= \lfloor v \cdot 10^{-k-1} \rfloor \\ &= \lfloor v \cdot 10^{-k-1} \cdot r_{1,-k-1} \rfloor \\ &= \lfloor v \cdot pow10 \cdot 2^{\lfloor (-k-1) \cdot \log 2(10) \rfloor - 63} \rfloor \\ &= \lfloor c \cdot pow10 \cdot 2^{q + \lfloor (-k-1) \cdot \log 2(10) \rfloor - 63} \rfloor \\ &= (c \cdot pow10) \gg (63 - q - \lfloor (-k-1) \cdot \log 2\,(10) \rfloor) \\ &= (c \cdot pow10) \gg (63 - h) \\ &= ((c \ll (37 + h)) \cdot pow10) \gg (36 + 64) \\ &= hi64 \gg 36 \end{aligned} \tag{189}$$

Suppose up is used to indicate whether one is equal to 10. Then, according to Equation (145) and Equation (112), there is

$$\begin{aligned} \text{bool } up &= \lfloor 2^{q+35} \cdot 10^{-k-1} \rfloor + even > 2^{36} - 1 - \lfloor 2^{36} \cdot n_r \rfloor \\ &= \lfloor 2^{q+35} \cdot 10^{-k-1} \rfloor + even + \lfloor 2^{36} \cdot n_r \rfloor \geqslant 2^{36} \\ &= (pow10 \gg (28 - h)) + even + \lfloor 2^{36} \cdot n_r \rfloor \geqslant 2^{36} \\ &= \left(half + \lfloor 2^{36} \cdot n_r \rfloor\right) \gg 36 \end{aligned} \tag{190}$$

When $one \neq 10$, $d//10$ is m. When $one = 10$, $d//10$ is equal to m+1. Then the following relationship exists:

$$d//10 = m + up \tag{191}$$

The high 28 bits of $hi64$ are represented as m, and the low 36 bits are represented as $\lfloor 2^{36} \cdot n_r \rfloor$.

$$hi64 = (m \ll 36) + \lfloor 2^{36} \cdot n_r \rfloor \tag{192}$$

Then the following relationship exists:

$$shorter = d//10 = (hi64 + half) \gg 36 \tag{193}$$

Suppose $updown$ is used to represent that one is equal to 0 or one is equal to 10, or it can be expressed as the last digit of d being 0.

$$updown = (d\%10 == 0) \tag{194}$$

To simplify the expression, let dot_one represent $\lfloor 2^{36} \cdot n_r \rfloor$. According to Equations (141) and (146), the following relationship holds:

$$\begin{aligned} one = 0 &: half > dot_one \\ one = 10 &: half > 2^{36} - 1 - dot_one \end{aligned} \tag{195}$$

The following equation holds true:

$$\begin{aligned} one = 0 &: (hi64 - half) \gg 36 = m - 1 \\ one = 10 &: (hi64 - half) \gg 36 = m \end{aligned} \tag{196}$$

Therefore, the following conclusion can be drawn:

$$\begin{aligned} one = 0 &: shorter = m > (hi64 - half) \gg 36 = m - 1 \\ one = 10 &: shorter = m + 1 > (hi64 - half) \gg 36 = m \end{aligned} \tag{197}$$

So when the following conditions are met, one equals 0 or one equals 10.

$$shorter > (hi64 - half) \gg 36 \Rightarrow d\%10 = 0 \tag{198}$$

When $updown$ is true, $d = 10 \cdot shorter$. When $updown$ is false, we need to calculate the value of d. From Equation (164), when $10n - \lfloor 10n \rfloor \neq 0.5$,it can be concluded that:

$$\begin{aligned} d &= \left(\lfloor 20 \cdot \left(v \cdot 10^{-k-1} \cdot r - n_{36}\right) \rfloor + 1\right) //2 \\ &= \left(\lfloor 20 \cdot \left(hi64 \cdot 2^{-36}\right) \rfloor + 1\right) //2 \\ &= \left(\lfloor 5 \cdot hi64 \cdot 2^{-34} \rfloor + 1\right) //2 \\ &= \left(\lfloor \left(5 \cdot hi64 + 2^{34}\right) \cdot 2^{-34} \rfloor\right) //2 \\ &= \left(\left(5 \cdot hi64 + 2^{34}\right) \gg 34\right) //2 \\ &= \left(5 \cdot hi64 + 2^{34}\right) \gg 35 \end{aligned} \tag{199}$$

In fact, we can consider adding 2^{34} as a rounding operation. When $10n - \lfloor 10n \rfloor = 0.5$, by using Equation (154) for calculation, it is only necessary to determine whether n is equal to 0.25. The process of calculating $longer$ seems difficult to explain, but the algorithm has passed the exhaustive test.

For double-precision floating-point numbers, the process of calculating d and k is as follows:

$$\begin{aligned} \text{uint64 } vi &= bit\ copy\ from\ \text{f64 } v \\ \text{uint64 } c, \text{int } q &= \ extract\ from\ vi \\ \text{const int } BIT &= 6 \\ \text{int } k &= (q \cdot 78913) \gg 18 \\ \text{int } h &= q + ((217707 \cdot (-k-1)) \gg 16) \\ \text{uint128 } pow10 &= \text{get_pow10}(-k-1) \\ \text{uint64 } cb &= c \ll (h + BIT + 1) \\ \text{uint128 } hi128 &= \text{umul64_128_hi128}\,(cb, pow10) \\ \text{bool } even &= (vi + 1)\ \&1 \\ \text{uint64 } half &= (pow10 \gg (64 - h)) + even \\ \text{uint64 } dot_one &= (\text{uint64})\,(hi128 \gg BIT) \\ \text{uint64 } ten &= 10 \cdot (hi128 \gg (BIT + 64)) \\ \text{uint64 } offset_num &= dot_one == 2^{62}?0 : 2^{63} + 6 \\ \text{uint64 } one &= ((\text{uint128})\ 10 \cdot dot_one + offset_num) \gg 64 \\ one &= half > dot_one?0 : one \\ one &= half > 2^{64} - 1 - dot_one?10 : one \\ \text{uint64 } d &= ten + one \end{aligned} \tag{200}$$

$pow10$ is calculated through equation (29), and there is the following relationship:

$$pow10 \cdot 2^{\lfloor (-k-1) \cdot \log 2(10) \rfloor - 127} = 10^{-k-1} \cdot r_{1,-k-1} \tag{201}$$

From equation (54), the following conclusion can be drawn:

$$\begin{aligned} m &= \lfloor v \cdot 10^{-k-1} \rfloor \\ &= \lfloor v \cdot 10^{-k-1} \cdot r_{1,-k-1} \rfloor \\ &= \lfloor v \cdot pow10 \cdot 2^{\lfloor (-k-1) \cdot \log 2(10) \rfloor - 127} \rfloor \\ &= \lfloor c \cdot pow10 \cdot 2^{q + \lfloor (-k-1) \cdot \log 2(10) \rfloor - 127} \rfloor \\ &= (c \cdot pow10) \gg (127 - q - \lfloor (-k-1) \cdot \log 2\,(10) \rfloor) \\ &= (c \cdot pow10) \gg (127 - h) \\ &= ((c \ll (7+h)) \cdot pow10) \gg (64 + 70) \\ &= hi128 \gg 70 \end{aligned} \tag{202}$$

The range of h is [-4, -1]. Since c contains a maximum of 53 bits of information, cb will not overflow. Therefore, $ten = 10 \cdot m$ can be accurately calculated. dot_one is equal to $\lfloor 2^{64} \cdot n_r \rfloor$. From Equation (112), it can be concluded that:

$$\begin{aligned} half &= (pow10 \gg (64 - h)) + even \\ &= \lfloor 2^{63} \cdot 2^{q} \cdot 10^{-k-1} \rfloor + even \end{aligned} \tag{203}$$

First, calculate the value of one. If there is a condition where one equals 0, replace one with 0. If there is a condition where one equals 10, replace one with 10. Calculate one using equation (172).

$$\begin{aligned} one &= \lfloor \frac{\lfloor 2^{64} \cdot n_r \rfloor}{2^{64}} \cdot 10 + \left((n = 0.25) ? 0 : \left(2^{-1} + \frac{6}{2^{64}} \right) \right) \rfloor \\ &= \left(dot_one \cdot 10 + \left(dot_one == 2^{62} ? 0 : 2^{63} + 6 \right) \right) \gg 64 \end{aligned} \tag{204}$$

In fact, there is another equivalent method to calculate one.

$$one = dot_one == 2^{62} ? 2 : \left(dot_one \cdot 10 + 2^{63} + 6 \right) \gg 64 \tag{205}$$

The determination of $one = 0$ is made through equation (134), and the determination of $one = 10$ is made through equation (145).

In the specific implementation of C/C++ code. The only branch instruction is that during the calculation of c and q, subnormal floating-point numbers will enter an unlikely branch. For the calculation process of normal floating-point numbers, there are no branch instructions, which reduces the cost of branch prediction failures. From the above pseudo-code implementation, it can be seen that the core implementation is very simple and concise. For the calculation process of irregular floating-point numbers, the code implementation places them in a separate branch for calculation. Since the number of irregular floating-point numbers is relatively small, it is a better option to calculate them in a separate branch.

3.8 decimal to string

The algorithm for printing floating-point numbers is divided into two parts. The content of the first part has been introduced previously. In this section, we will introduce the second part.

Floating-point number printing usually includes two formats: fixed-point notation and scientific notation. Some examples are shown in Table (1) .

Table 1. Example of floating-point number printing result.

float number	fixed-point	scientific
2.34	"2.34"	"2.34e+00"
12	"12.0"	"1.2e+01"
120	"120.0"	"1.2e+02"
0.012	"0.012"	"1.2e-02"

We have already provided a detailed description of the process of calculating d and k previously. We need to convert the floating-point numbers into strings based on d and k. In fact, according to the Schubfach algorithm, d may contain trailing zeros, meaning that $d\%10$ could be 0. We divide d into two parts: $d//10$ and $d\%10$. We use up to indicate whether one is equal to 10, and $updown$ to represent the cases where one is either 0 or 10. So the following relationship exists:

$$\begin{aligned} d//10 &= m + up \\ updown &= (one == 0 \text{ or } 10) = (d\%10 == 0) \\ d &= 10 \cdot (m + up) + (updown?0 : one) \end{aligned} \tag{206}$$

By calculating the approximate range of m, we can conclude that:

$$\begin{aligned} m &= \lfloor c \cdot 2^q \cdot 10^{-k-1} \rfloor \\ c \cdot 2^q \cdot 10^{-k-1} &\in [0.1 \cdot c, c) \\ \Rightarrow \lfloor 0.1 \cdot c_{\min} \rfloor \leqslant \lfloor 0.1 \cdot c \rfloor \leqslant & m \leqslant \lfloor c \rfloor \leqslant \lfloor c_{\max} \rfloor \end{aligned} \tag{207}$$

Based on the range of c, it can be concluded that:

$$\begin{aligned} float &: \begin{cases} \text{normal} : \left[\frac{2^{23}+1}{10}, 2^{24} - 1\right] = [838860.9, 16777215] \\ \text{subnormal} : \left[\frac{1}{10}, 2^{23} - 1\right] = [0.1, 8388607] \end{cases} \\ double &: \begin{cases} \text{normal} : \left[\frac{2^{52}+1}{10}, 2^{53} - 1\right] = [4.5e14, 9e15] \\ \text{subnormal} : \left[\frac{1}{10}, 2^{52} - 1\right] = [0.1, 4.5e15] \end{cases} \end{aligned} \tag{208}$$

Let $len10(x)$ represent the number of digits in the decimal number x. When $x > 0$, then the following relationship exists:

$$len10\,(x) = \lfloor \log_{10}(x) \rfloor + 1 \tag{209}$$

Special definition: $len10(0) = 0$. Here is an example: $len10(12345) = 5$. Therefore, there is:

$$\begin{aligned} float &: \begin{cases} \text{normal} : len10\,(m + up) \in [6, 8] \\ \text{subnormal} : len10\,(m + up) \in [0, 7] \end{cases} \\ double &: \begin{cases} \text{normal} : len10\,(m + up) \in [15, 16] \\ \text{subnormal} : len10\,(m + up) \in [0, 16] \end{cases} \end{aligned} \tag{210}$$

And there is a relationship of $len10(d) = len10(m + up) + 1$ because of $d//10 = m + up$. We denote the number of trailing zeros in the decimal number x as $tz10(x)$, and here is an example: $tz10(12300) = 2$. Then, the number of significant digits of the decimal number x, denoted as $sig10(x)$, is equal to $len10(x) - tz10(x)$. When $updown$ is 0 , $one \neq 0$, then $d\%10 = one \neq 0$,

and $tz10(d) = 0$. Then the following conclusion can be drawn:

$$\begin{aligned} sig10\,(d) &= updown?len10\,(d) - tz10\,(d) : len10\,(d) \\ float :& sig10\,(d) \in [1, 9] \\ double :& sig10\,(d) \in [1, 17] \end{aligned} \tag{211}$$

For the common normal floating-point numbers, the calculation method of $len10(d)$ is as follows:

$$\begin{aligned} float : len10\,(d) &= len\,(m + up) + 1 = 9 - \left(m + up < 10^7\right) - \left(m + up < 10^6\right) \\ double : len10\,(d) &= len\,(m + up) + 1 = 16 + \left((m + up) \geqslant 10^{15}\right) \end{aligned} \tag{212}$$

Calculating the ASCII code of d can be transformed into calculating the ASCII codes of $m+up$ and one. The following relationship exists:

$$\begin{aligned} float &: m + up < 10^8 \\ double &: m + up < 10^{16} \end{aligned} \tag{213}$$

Due to the existence of the above relationship, calculating the ASCII code of $m + up$ can be easily accomplished by utilizing the SIMD instruction set of the CPU.

First, we introduce scalar method that does not rely on SIMD instructions. Suppose that $x \in \left[0, 10^8\right)$, and $dec_to_ascii8(x)$ is a function that calculates the ASCII value of x. The calculation process of $dec_to_ascii8(x)$ is as shown in Algorithm (1). Therefore, when calculating the ASCII

Algorithm 1 Convert an 8 digits decimal number to an ASCII code : dec_to_ascii8(x)

Input: X(type:uint64 , range:[0,1e8))
Output: Y(type:uint64) , tz(type:uint)

```
uint64 abcd_efgh=X+(0x100000000-10000)*((X*0x68db8bbULL)»40)
uint64 ab_cd_ef_gh=abcd_efgh+(0x10000-100)*(((abcd_efgh*0x147b)»19)&0x7f0000007f)
uint64 a_b_c_d_e_f_g_h=ab_cd_ef_gh+(0x100-10)*(((ab_cd_ef_gh*0x67)»10)&0xf000f000f000f)
uint tz=__builtin_ctzll(a_b_c_d_e_f_g_h)»3
uint64 BCD = CPU_IS_LITTLE_ENDIAN ? byteswap(a_b_c_d_e_f_g_h) : a_b_c_d_e_f_g_h
uint64 Y = BCD + 0x3030303030303030
return Y,tz
```

value of x, $tz10(x)$ can be easily calculated. $tz10(x)$ is equal to the return value tz.

Suppose that $x \in \left[0, 10^{16}\right)$, and $dec_to_ascii16(x)$ is a function that calculates the ASCII value of x. The calculation process of $dec_to_ascii16(x)$ is as shown in Algorithm (2).

Algorithm 2 Convert a 16 digits decimal number to an ASCII code : dec_to_ascii16(x)

Input: X(type:uint64 , range:[0,1e16))
Output: Y(type:uint128) , tz(type:uint)

```
uint64 abcdefgh=X//10^8
uint64 ijklmnop=X%10^8
(uint64 abcdefgh_ascii, uint tz1)=dec_to_ascii8(abcdefgh)
(uint64 ijklmnop_ascii, uint tz2)=dec_to_ascii8(ijklmnop)
uint128 Y = (abcdefgh_ascii«64) + ijklmnop_ascii
Y = CPU_IS_LITTLE_ENDIAN ? (ijklmnop_ascii«64) + abcdefgh_ascii : Y
uint tz = (ijklmnop == 0) ? 8 + tz1 : tz2
return Y,tz
```

Since $__builtin_ctzll(0)$ is an undefined behavior in C language. We need to carry out special processing. For single-precision floating-point numbers, when $m + up$ equals 0(the six smallest subnormal floating-point numbers), it can be concluded that $updown$ is 0. Therefore, the use of the value of $__builtin_ctzll(0)$ can be avoided. For double-precision floating-point numbers, when $m + up$ equals 0, this condition exists only when the floating-point number is 5e-324(the smallest subnormal floating-point number). Therefore, when the floating-point number is greater than 5e-324, $m + up > 0$, and the use of the value of $__builtin_ctzll(0)$ can also be avoided. In the code implementation, we handle the special value 5e-324 separately at the beginning of the function.

In the SIMD implementation of the dec_to_ascii8 and $dec_to_ascii16$ algorithms., for the ARM64 architecture, the NEON instruction set is adopted; for the x64 architecture, three different methods, namely AVX512, SSE4.1, and SSE2, are used. The implementation principle of SIMD is also based on the aforementioned scalar method. Due to space limitations, we will not provide extensive demonstrations of the relevant implementation. The table (2) below provides a brief description of the SIMD implementation.

Table 2. SIMD implementation of the dec_to_ascii8 and $dec_to_ascii16$ algorithms

SIMD implementation	description
NEON	Original author: Dougall Johnson. It runs on an ARM processor and requires the NEON instruction set.
SSE2	Based on the scalar version, only the SSE2 instruction set is required.
SSE4.1	It is almost the same as the SSE2 implementation. The SSE4.1 instruction set is required.
AVX512	Original author: Daniel Lemire. This implementation requires the AVX512IFMA and AVX512VBMI instruction set.

Through the above methods, the ASCII value of $m + up$ and the number of significant digits can be calculated. Printing d is equivalent to printing $m + up$ and one. By using $sig10(d)$, it is possible to determine which contents need to be retained in the buffer. In the code implementation, we adopt different formats for floating-point numbers within different ranges to enhance readability.

Table 3. Printing formats for different ranges

type	fixed-point	scientific
float	[1e-3,1e7)	other range
double	[1e-4,1e16)	other range

In scientific notation format, there are exponential result included. For example, in "1.2e-02", "e-02" represents the exponential result. Convert $d \cdot 10^k$ to the standard decimal scientific notation result, as follows:

$$d \cdot 10^k = \left(d \cdot 10^{-\lfloor \log_{10}(d) \rfloor}\right) \cdot 10^{k+\lfloor \log_{10}(d) \rfloor} \tag{214}$$

The range of $\left(d \cdot 10^{-\lfloor \log_{10}(d) \rfloor}\right)$ is within the interval $[1, 10)$. Therefore, the exponential result can be determined based on $k + \lfloor \log_{10}(d) \rfloor$. Assume that $E_{10} = k + \lfloor \log_{10}(d) \rfloor$,we have:

$$E_{10} = k + \lfloor \log_{10}(d) \rfloor = k + \lfloor \log_{10}(m + up) \rfloor + 1 \tag{215}$$

For single-precision floating-point numbers, when $E_{10} \in [-3, 6]$, fixed-point notation is adopted. For double-precision floating-point numbers, when $E_{10} \in [-4, 15]$, fixed-point notation is adopted.

When converting $m + up$ to ASCII values, the algorithm implemented in this paper utilizes the SIMD instruction set, which is the key difference from other algorithms. For float, the dec_to_ascii8

Algorithm 3 A simple description of the floating-point number printing algorithm

Input: v(type:float/double),buffer(type:char*)
Output: buffer(type:char*)

1: compute "m+up,updown,one,k" from v
2: compute sig10(d)=updown ? len10(m+up)-tz10(m+up) : len10(m+up)+1
3: compute E_{10}
4: convert "m+up,one" to ASCII code and store them in the buffer.
5: based on sig10(d) and E_{10}, it is determined which contents in the buffer should be retained.
6: If the result is in scientific notation, print E_{10}.
7: **return** buffer

algorithm is used; for double, the $dec_to_ascii16$ algorithm is employed. For the detailed implementation, you need to read the source code. In the C code implementation, all the branches are unlikely branches to minimize the impact of branch prediction failures. The process of calculating d is transformed into calculating $m + up$ and one, which reduces instruction dependencies and enhances instruction-level parallelism. There is no need to calculate the exact value of d. Instead, we can quickly calculate $d//10 = m+up$. However, other algorithms always calculate the accurate value of d first and then print it.

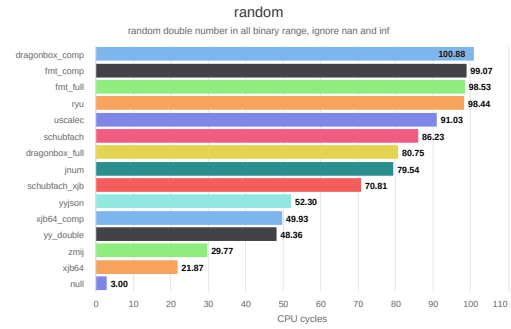
Table 4. All algorithms in the benchmark test.

algorithm	float	double	description
Schubfach	Schubfach32	Schubfach64	author:Raffaello Giulietti,https://github.com/c4f7fcce9cb06515/Schubfach.
Schubfach_xjb	Schubfach32_xjb	Schubfach64_xjb	It is improved by Schubfach and has the same output result.
Ryu	Ryu32	Ryu64	author:Ulf Adams,https://github.com/ulfjack/ryu.
Dragonbox	Dragonbox32	Dragonbox64	author:Junekey Jeon,https://github.com/jk-jeon/Dragonbox.
fmt[10]	fmt32	fmt64	author:Victor Zverovich,https://github.com/fmtlib/fmt version:12.1.0
yy_double	-	yy_double	author:GuoYaoYuan,link:yy_double.
yy_json[11]	yy_json32	yy_json64	author:Guo YaoYuan,https://github.com/ibireme/yyjson version:0.12.0
teju_jagua[12]	teju32	teju64	author:Cassio Neri,https://github.com/cassioneri/teju_jagua.
xjb	xjb32	xjb64	this paper,https://github.com/xjb714/xjb.
zmij[13]	zmij32	zmij64	author:Victor Zverovich,https://github.com/vitaut/zmij.
jnum[14]	jnum32	jnum64	author:Jing Leng,https://github.com/lengjingzju/json/jnum.c.
uscalec[15]	-	uscalec	author:Russ Cox,src:https://github.com/rsc/fpfmt commit 6255750 (19 Jan 2026).

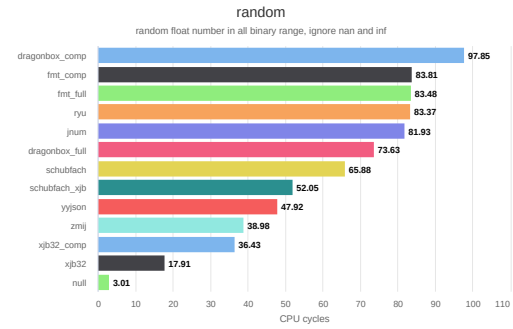
4 BENCHMARK RESULT

For single-precision floating-point numbers, the two processes of converting the floating-point number to a decimal result and a string have been tested with all possible input values compared to the schubfach algorithm. For double-precision floating-point numbers, we have verified their correctness through a complete random number test.

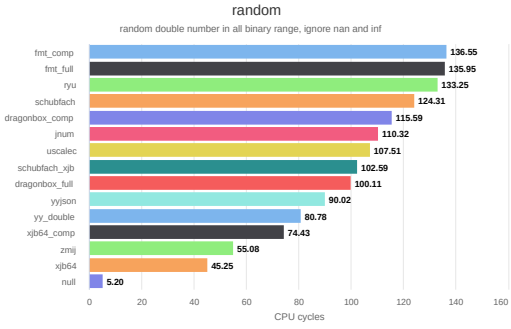
In the performance test comparison, we compared the time spent by the following several different algorithms converting floating-point numbers to decimal results and string, as shown in Table (4). Test process: Generate 2^{24} random numbers without 0, NaN, and Inf, measure the total time spent converting all floating-point numbers to decimal and string results, and obtain the average time for converting a single floating-point number to decimal and string. The compiler used for AMD R7-7840H is icpx 2025.0.4, and the compiler used for Apple M1 is apple clang 17.0.0. The



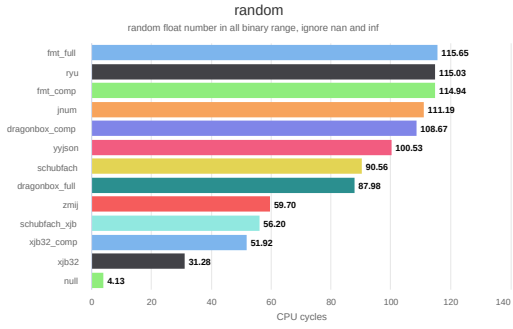
(a) Apple M1 double benchmark result



(b) Apple M1 float benchmark result



(c) AMD R7-7840H double benchmark result



(d) AMD R7-7840H float benchmark result

Fig. 1. float/double to string benchmark results

compilation option for all compilers is ”-O3 -march=native”. We conducted benchmark tests on two processors, and the test results are shown in Fig (1a) to Fig (1d) ,table (5) and table (6).

Special note: The algorithm of teju_jagua only supports float/double to decimal, because its author did not implement the source code of decimal to string. yy_double only supports double. Dragonbox_comp,fmt_comp and xjb_comp represent the versions of the compressed constant lookup table. Dragonbox_full and fmt_full represent uncompressed constant lookup table.

From the benchmark results, it can be seen that the performance of the algorithm in this paper is better than other algorithms in most cases.

Since all the branches in the algorithm code implementation of this article are unlikely branches, the performance remains almost constant regardless of the different input values. Even when the input values are completely random, it still maintains a very high branch prediction success rate.

5 CONCLUSIONS

This paper proposes a new floating-point number to string conversion algorithm. The algorithm improves the calculation process of Schubfach algorithms, reduces the number of multiplication operations, and optimizes some calculation steps. The algorithm has been implemented in C/C++ language and passed exhaustive tests. The benchmark results show that the performance of the algorithm is better than most existing algorithms in most cases.

Table 5. float/double to decimal benchmark results on AMD R7-7840H and ubuntu 26.04. The unit is nanosecond(ns).

algorithm	float	double
	icpx 2025.0.4	icpx 2025.0.4
Schubfach	12.2	11.51
Schubfach_xjb	4.44	6.33
Ryu	14.02	13.08
Dragonbox	10.19	10.05
yy_json	4.67	5.72
yy_double	-	5.24
teju_jagua	14.99	14.37
zmij	4.76	4.78
uscalec	-	11.27
xjb	2.24	3.76

Table 6. float/double to decimal benchmark results on Apple M1 and MacOS 26.3.1. The unit is nanosecond(ns).

algorithm	float	double
	apple clang 17.0.0	apple clang 17.0.0
Schubfach	11.64	13.12
Schubfach_xjb	5.16	6.58
Ryu	15.75	14.16
Dragonbox	11.78	12.03
yy_json	3.97	4.46
yy_double	-	4.08
teju_jagua	20.25	18.66
zmij	4.11	3.83
uscalec	-	15.26
xjb	2.15	2.58

REFERENCES

[1] Guy L. Steele and Jon L. White. 1990. How to print floating-point numbers accurately. *SIGPLAN Not.* 25, 6 (June 1990), 112–126. doi:10.1145/93548.93559

[2] Florian Loitsch. 2010. Printing floating-point numbers quickly and accurately with integers. *SIGPLAN Not.* 45, 6 (June 2010), 233–243. doi:10.1145/1809028.1806623

[3] Marc Andrysco, Ranjit Jhala, and Sorin Lerner. 2016. Printing floating-point numbers: a faster, always correct method. *SIGPLAN Not.* 51, 1 (Jan. 2016), 555–567. doi:10.1145/2914770.2837654

[4] Ulf Adams. 2018. Ryū: fast float-to-string conversion. *SIGPLAN Not.* 53, 4 (June 2018), 270–282. doi:10.1145/3296979.3192369

[5] Ulf Adams. 2019. Ryū revisited: printf floating point conversion. *Proc. ACM Program. Lang.* 3, OOPSLA, Article 169 (Oct. 2019), 23 pages. doi:10.1145/3360595

[6] R. Giulietti. 2020. The Schubfach Way to Render Doubles. (Sept. 2020). https://drive.google.com/file/d/1KLtG_LaIbK9ETXI290zqCxvBW94dj058/view

[7] Junekey Jeon. 2020. Grisu-Exact: A Fast and Exact Floating-Point Printing Algorithm. (Sept. 2020). https://github.com/jk-jeon/Grisu-Exact/blob/master/other_files/Grisu-Exact.pdf.

[8] Junekey Jeon. 2024. Dragonbox: A New Floating-Point Binary-to-Decimal Conversion Algorithm. (July 2024). https://github.com/jk-jeon/dragonbox/blob/master/other_files/Dragonbox.pdf

[9] Guo YaoYuan. 2024. (Nov. 2024). https://github.com/ibireme/c_numconv_benchmark/blob/master/vendor/yy_double/yy_double.c
[10] Victor Zverovich. 2025. (Oct. 2025). https://github.com/fmtlib/fmt
[11] Guo YaoYuan. 2025. (Aug. 2025). https://github.com/ibireme/yyjson
[12] Cassio Neri. 2025. (Nov. 2025). https://github.com/cassioneri/teju_jagua
[13] Victor Zverovich. 2026. (Jan. 2026). https://github.com/vitaut/zmij
[14] Jing Leng. 2025. (Nov. 2025). https://github.com/lengjingzju/json/jnum.c
[15] Russ Cox. 2026. (Jan. 2026). https://github.com/rsc/fpfmt

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009